

# CurveConstruction

Full source listing of the vintage\_curves package, examples, and tests — ordered for reading.

## Reading order

1. README.md
2. VINTAGE\_CURVES\_PLAN.md
3. pyproject.toml
4. vintage\_curves/\_\_init\_\_.py
5. vintage\_curves/schema.py
6. vintage\_curves/tape.py
7. vintage\_curves/mob.py
8. vintage\_curves/default\_policy.py
9. vintage\_curves/metrics.py
10. vintage\_curves/segment.py
11. vintage\_curves/aggregate.py
12. vintage\_curves/rolls.py
13. vintage\_curves/summary.py
14. vintage\_curves/plotting.py
15. vintage\_curves/output.py
16. vintage\_curves/analyzer.py
17. examples/README.md
18. examples/schema.yaml
19. examples/generate\_sample\_tape.py
20. examples/run\_analysis.py
21. tests/\_\_init\_\_.py
22. tests/test\_schema\_and\_tape.py
23. tests/test\_mob.py
24. tests/test\_default\_policy.py
25. tests/test\_metrics.py
26. tests/test\_segment.py
27. tests/test\_aggregate.py
28. tests/test\_rolls.py
29. tests/test\_summary.py
30. tests/test\_end\_to\_end.py

# README.md

## CurveConstruction

`vintage_curves` is a Python package that turns a consumer-loan performance tape (one row per loan-month) into vintage-level performance curves — cumulative loss, prepay, SMM/CPR, MDR/CDR, pool factor, WAC — plus aggregate curves across vintages (three weighting schemes), per-vintage summary scalars, and roll-rate transition matrices.

The design is documented in `[VINTAGE_CURVES_PLAN.md](VINTAGE_CURVES_PLAN.md)`.

## Quick start

```
from vintage_curves import VintageAnalyzer, DefaultPolicy

va = VintageAnalyzer.from_csv(
    "examples/sample_tape.csv",
    schema="examples/schema.yaml",
    vintage_freq="Q",
    default_policy=DefaultPolicy.charge_off_field(),
)

seg = va.segment("fico >= 720", label="prime+")

seg.curve("cum_gross_loss")                # wide vintage x MOB
seg.aggregate(["cum_gross_loss", "cpr", "cdr"], # MOB x metric (+ n_vintages)
              weighting="original")
seg.summary()                             # per-vintage scalars
seg.roll_rates(by_mob=False)              # transition matrix
seg.plot("cum_gross_loss", include_agg=True).savefig("loss.png")
seg.export_excel("analysis.xlsx")
```

Or run the worked example end-to-end:

```
pip install -e .
python3 examples/run_analysis.py
```

## Install

```
pip install -e .
```

Python 3.11+. Depends on `pandas`≥2, `numpy`, `pyyaml`, `matplotlib`, `openpyxl`.

## Tests

```
pip install -e ".[test]"
pytest
```

The full suite includes a hand-derived analytical fixture for every metric plus end-to-end coverage on the bundled 749k-row synthetic tape.

# VINTAGE\_CURVES\_PLAN.md

## Vintage Performance Curves — Plan of Action

A Python library for taking a consumer-loan performance tape (loan + borrower characteristics + monthly performance data) and producing vintage-level performance curves, with arbitrary segmentation and standard ABS metrics.

---

### 1. Scope & Goals

**In scope (v1):** - Single flat CSV input (one row per loan-month). - User-supplied column-name mapping (YAML or dict) → canonical schema. - Arbitrary pandas-query( ) style segmentation strings. - Configurable default definition: Charge-Off Date field **OR** a DPD threshold (sticky — once defaulted, stays defaulted even on cure). - Monthly, quarterly, **or** annual vintage grouping, anchored on origination date. - MOB = months since origination. - Per-vintage curves **and** aggregate curves across vintages, with three weighting schemes (original balance / beginning balance at MOB / equal). Each aggregate is published alongside an n\_vintages series so consumers can see where support collapses. - Truncate each vintage at its maximum observable MOB. - Wide matrix output (vintage × MOB) per metric, plus matplotlib plotting and CSV/Excel export. - Portfolio summary stats per vintage (WA Coupon, Pool Factor, WAL). - Delinquency transition / roll-rate matrices. - Unit tests against a small synthetic tape.

**Out of scope (v1):** - Multi-file inputs, Parquet/Excel readers (can add later via a thin loader layer). - CLI / dashboard. - Net loss curves (gross only for v1; explicitly deferred). - Polars / DuckDB backends. Code will be structured so the engine could be swapped, but v1 ships pandas only.

---

### 2. Canonical Schema

All internal logic operates on canonical column names. Users provide a mapping from their tape's columns to these names.

#### ***Loan characteristics (one value per Loan\_ID, constant over time)***

|           |                  |                        |             |                                 |     |                            |                  |        |                        |
|-----------|------------------|------------------------|-------------|---------------------------------|-----|----------------------------|------------------|--------|------------------------|
| Canonical | Type             | Notes                  | --- --- --- | loan_id                         | str | Primary key                | origination_date | date   | Used to derive vintage |
|           | original_balance | float                  |             | Denominator for cum loss/prepay |     | origina <del>l</del> _term | int              | Months | apr                    |
|           | float            | Decimal (e.g., 0.1499) |             | product_type                    | str | Optional                   | deferral_period  | int    | Optional, months       |

#### ***Borrower characteristics (one value per Loan\_ID)***

|           |      |  |         |      |     |  |     |       |  |        |       |  |       |     |  |
|-----------|------|--|---------|------|-----|--|-----|-------|--|--------|-------|--|-------|-----|--|
| Canonical | Type |  | --- --- | fico | int |  | dti | float |  | income | float |  | state | str |  |
|-----------|------|--|---------|------|-----|--|-----|-------|--|--------|-------|--|-------|-----|--|

#### ***Performance (one row per loan-month)***

|           |                     |       |             |                               |      |                              |             |                                    |       |                           |
|-----------|---------------------|-------|-------------|-------------------------------|------|------------------------------|-------------|------------------------------------|-------|---------------------------|
| Canonical | Type                | Notes | --- --- --- | as_of_date                    | date | Month-end of the observation |             |                                    |       |                           |
|           | outstanding_balance | float |             | Period-end                    |      | scheduled_principal          | float       | Scheduled principal for the period |       |                           |
|           | principal_payment   | float |             | Actual principal paid (total) |      | interest_payment             | float       | Actual interest paid               |       |                           |
|           | fees_paid           | float |             | Optional                      |      | days_past_due                | int         | DPD at period end                  |       |                           |
|           |                     |       |             |                               |      | charge_off_date              | date or NaT |                                    |       |                           |
|           |                     |       |             |                               |      | Filled when CO occurs        |             | charge_off_amount                  | float | Loss amount at CO (gross) |

(Recoveries / recovery\_amount deliberately omitted from v1 — net loss is out of scope.)

Mapping config example (schema.yaml):

```
loan_id: LoanNumber
origination_date: OrigDt
```

```

original_balance: OrigUPB
apr: NoteRate
fico: BorrowerFICO
as_of_date: ReportingPeriod
outstanding_balance: EndingBalance
charge_off_date: ChargeOffDt
charge_off_amount: ChargeOffAmt
days_past_due: DPD
principal_payment: PrincipalPaid
interest_payment: InterestPaid

```

A SchemaMapper class loads the YAML, renames columns, validates required fields are present, and coerces dtypes.

---

### 3. Architecture

```

vintage_curves/
■■■■ __init__.py
■■■■ schema.py           # SchemaMapper, canonical column constants, required-field validation
■■■■ tape.py             # LoanTape: load CSV, apply schema, split into loan-level vs perf-level frames
■■■■ default_policy.py   # DefaultPolicy: charge_off_field | dpd_threshold(n)
■■■■ segment.py          # Segment: query string + label; SegmentSet for batches
■■■■ mob.py              # add_mob(perf, loans) -> adds 'mob' and 'vintage' columns
■■■■ metrics.py          # Per-vintage metric calcs (cum gross loss, cum prepay, SMM/CPR, CDR, etc.)
■■■■ aggregate.py        # Balance-weighted aggregation across vintages
■■■■ summary.py          # WA Coupon, Pool Factor, WAL per vintage
■■■■ rolls.py            # Delinquency bucket assignment + transition matrices
■■■■ output.py           # Wide-matrix builder, CSV/Excel export
■■■■ plotting.py         # plot_curves(matrix, title=...) matplotlib helpers
■■■■ analyzer.py         # VintageAnalyzer – the public façade
tests/
■■■■ synthetic.py        # Generates a small deterministic tape
■■■■ test_metrics.py
■■■■ test_segment.py
■■■■ test_rolls.py
■■■■ test_end_to_end.py

```

#### Public API (sketch)

```

from vintage_curves import VintageAnalyzer, DefaultPolicy

va = VintageAnalyzer.from_csv(
    "tape.csv",
    schema="schema.yaml",
    vintage_freq="Q",                # "M", "Q", or "A"
    default_policy=DefaultPolicy.charge_off_field(), # or .dpd_threshold(120)
)

# Segment
seg = va.segment("fico >= 720 and fico <= 759 and original_term == 120",
    label="720-759 / 120mo")

# Per-vintage curves: rows = vintages, cols = MOB
cum_gross_loss = seg.curve("cum_gross_loss")    # DataFrame (wide)
cpr             = seg.curve("cpr")
cdr             = seg.curve("cdr")
pool_factor     = seg.curve("pool_factor")

# Aggregate across vintages – one row per MOB
# weighting: "original" (default) weights each vintage by its original balance;
#             "beginning" uses begin-of-period balance at each MOB;
#             "equal" simple-averages across vintages (sanity check).
# Every aggregate frame includes an `n_vintages` column showing support at each MOB.
agg_orig = seg.aggregate(metrics=["cum_gross_loss", "smm", "cdr"], weighting="original")
agg_bop  = seg.aggregate(metrics=["cum_gross_loss", "smm", "cdr"], weighting="beginning")
agg_eq   = seg.aggregate(metrics=["cum_gross_loss", "smm", "cdr"], weighting="equal")

# Per-vintage summary stats
summary = seg.summary()          # wa_coupon, pool_factor_latest, wal, original_count, original_bal

# Roll-rate transition matrix (current→30→60→90→C0)

```

```

rolls = seg.roll_rates(by_mob=True)

# Plot & export
seg.plot("cum_gross_loss").savefig("loss.png")
seg.export_excel("output.xlsx") # one sheet per metric + summary + rolls

---
```

## 4. Metric Definitions

Let original balance for vintage  $v$  be  $OB_v = \sum \text{original\_balance}$  for loans in  $v$ . Let  $bal_{\{l, t\}}$  be outstanding (end-of-period) balance for loan  $l$  at MOB  $t$ .

**Beginning-of-period (BOP) balance** is defined as the prior period's EOP balance, with `original_balance` substituted at MOB 1 (and MOB 0 treated as the origination snapshot). If the tape has a gap for a loan-month, BOP for the next observed month falls back to the most recent prior EOP; gaps are flagged in a load-time diagnostic.

**Performing-loan set** at MOB  $t$  for vintage  $v$ : all loans in  $v$  that have not defaulted (per the active `DefaultPolicy`) and have not fully prepaid as of the start of period  $t$ . Defaults are **sticky**: once a loan defaults, it is removed from the performing set for all subsequent MOBs regardless of any later cure.

### Per-vintage curves

- **Cumulative gross loss** %  $cum\_gross\_loss_{\{v, t\}} = (\sum_{\{l \in v, mob \leq t\}} \text{charge\_off\_amount}_l) / OB_v$   
- **Cumulative prepayments** %  $cum\_prepay_{\{v, t\}} = (\sum \text{unscheduled principal}) / OB_v$  where `unscheduled` = `principal_payment` - `scheduled_principal` (floored at 0), summed **only over performing loan-months** — the default month and all later months for a defaulted loan are excluded from the prepay numerator so charge-off write-downs are never counted as prepays.  
- **SMM (Single Monthly Mortality)** at MOB  $t$  for vintage  $v$ , restricted to performing loans:  $SMM_{\{v, t\}} = \text{unscheduled\_principal}_{\{v, t\}} / (\text{BOP\_bal}_{\{v, t\}} - \text{scheduled\_principal}_{\{v, t\}} - \text{default\_bal}_{\{v, t\}})$  where  $\text{default\_bal}_{\{v, t\}}$  is the BOP balance of loans that default during period  $t$  (PSA convention — keeps prepay rate uncontaminated by defaults).  
- **CPR (annualized)**  $CPR_{\{v, t\}} = 1 - (1 - SMM_{\{v, t\}})^{12}$   
- **MDR / CDR (annualized)**  $MDR_{\{v, t\}} = \text{default\_bal}_{\{v, t\}} / \text{BOP\_bal\_performing}_{\{v, t\}}$ , then  $CDR_{\{v, t\}} = 1 - (1 - MDR_{\{v, t\}})^{12}$ . Denominator is the BOP **actual** balance of performing loans for vintage  $v$  (same set used for SMM, before the scheduled-principal carve-out).  
- **Pool Factor**  $\text{pool\_factor}_{\{v, t\}} = (\sum bal_{\{l, t\}}) / OB_v$   
- **WA Coupon** balance-weighted apr at MOB  $t$  (matrix or single per-vintage value)  
- **WAL** per vintage, in months. Two flavors are emitted side-by-side because realized-only WAL is biased downward by defaults/prepays:  
- **wal\_realized**:  $\sum t \cdot \text{principal\_received}_t / \sum \text{principal\_received}_t$  over observed history.  
- **wal\_scheduled**: same formula but using the **scheduled** principal stream (level-pay implied by `original_balance`, `apr`, `original_term`), ignoring defaults and prepays.

### Default policy

- `DefaultPolicy.charge_off_field()`: a loan defaults in the month its `charge_off_date` falls; `loss` = `charge_off_amount`.  
- `DefaultPolicy.dpd_threshold(n)`: a loan defaults the first MOB where `days_past_due`  $\geq n$ ; `loss` = **EOP outstanding\_balance of the default month** (gross).  
- **Stickiness**: under either policy, once a loan is flagged defaulted it stays defaulted for all subsequent MOBs even if it later cures. The loan is removed from performing denominators (SMM, CDR, pool factor performing-share) from the default month forward.  
- **Coexistence of fields**: if the user selects `dpd_threshold(n)` but the tape **also** populates `charge_off_date` on some loans, schema validation raises by default. The user can pass `on_conflict="charge_off_wins"` or `"dpd_wins"` to resolve explicitly; the resolution choice is logged.

### Aggregate across vintages (three weighting schemes)

For metric  $M$  at MOB  $t$ , restricted to vintages where MOB  $t$  is observable:

- **weighting="original" (default)** — weight each vintage by its original balance  $OB_v$ . - Stock metrics (cum loss %, cum prepay %, pool factor):  $M_{agg\_t} = \sum_v OB_v \cdot M_{\{v, t\}} / \sum_v OB_v$ . - Flow / rate metrics (SMM, CPR, MDR, CDR): aggregate raw \$ numerator and \$ denominator separately, then form the ratio — equivalent to dollar-weighting by original balance contributions. - Interpretation: "the average originated dollar's lifetime experience" — keeps each vintage's footprint proportional to how big it was at origination.

- **weighting="beginning"** — weight each vintage at MOB  $t$  by its **beginning-of-period balance** at that MOB. - Stock metrics:  $M_{agg\_t} = \sum_v BOP_{\{v, t\}} \cdot M_{\{v, t\}} / \sum_v BOP_{\{v, t\}}$ . - Flow / rate metrics: numerator =  $\sum_v flow_{\{v, t\}}$ ; denominator =  $\sum_v BOP_{\{v, t\}}$  (or  $BOP - scheduled\_principal$  for SMM, consistent with the per-vintage definition). - Interpretation: "the average outstanding dollar's instantaneous behavior" — reweights toward vintages that still have balance at MOB  $t$ . Useful for current-portfolio behavior; CPR/CDR comparisons can differ noticeably from the original-weighted view late in a curve.

- **weighting="equal"** — simple unweighted average of  $M_{\{v, t\}}$  across vintages observable at MOB  $t$ . Useful as a sanity check against original-weighted aggregates that one large vintage can dominate.

All three views are always available; Excel export emits each as a separate sheet. Every aggregate frame carries an `n_vintages` column (count of vintages contributing at each MOB) so users can see exactly where support thins out.

### Truncation

For each vintage  $v$  with `last as_of_date = T_v`, the maximum observable MOB is  $(T_v - origination\_month_v)$  in months. Cells beyond `mob_max_v` are NaN (and dropped from aggregate denominators for that MOB).

### Delinquency buckets & roll rates

Buckets: Current (DPD=0–29), 30–59, 60–89, 90+, C0. (Standard consumer-credit convention folds 1–29 DPD into Current; the bucket boundaries are configurable on `roll_rates(...)` for users who want finer-grained early-stage tracking.) Transition matrix: count or balance share moving from bucket  $i$  (MOB  $t$ ) → bucket  $j$  (MOB  $t+1$ ), normalized within row. Optionally per-MOB or pooled.

---

## 5. Implementation Notes

- **Engine:** pandas. Group operations are vectorized; one merge of loan-level attrs onto perf at load time, then group by vintage / mob. - **MOB:** `mob = (as_of_date.year - orig.year) * 12 + (as_of_date.month - orig.month)`. - **Vintage:** `vintage = orig.to_period("M", "Q", or "A")` (calendar quarters/years, not fiscal). - **BOP balance:** derived once at load time as the prior period's EOP `outstanding_balance` per loan, with `original_balance` used at MOB 1. Stored as a column `bop_balance` on the perf frame. Gaps in the per-loan monthly sequence are flagged in a load diagnostic. - **Memory hook:** load function accepts `usecols`, `dtype`, and `chunksize` parameters as future swap-in points (documented, not exercised in v1). - **Required columns** (hard-fail in schema validation if missing or all-NaN): loan-level — `loan_id`, `origination_date`, `original_balance`, `original_term`, `apr`; perf-level — `as_of_date`, `outstanding_balance`, `scheduled_principal`, `principal_payment`, `days_past_due`. No silent fallback to a computed level-pay schedule for `scheduled_principal`; users needing that must precompute the column. - **Sign conventions:** payments positive, balances non-negative. - **Plotting:** each `plot_curves` returns a `matplotlib.figure.Figure`. One line per vintage; aggregate overlaid in bold black when `include_agg=True`.

---

## 6. Synthetic Test Tape

tests/synthetic.py generates a deterministic tape: - 3 vintages × 50 loans each, 60-month term, 12% APR. - Configurable default rate per vintage to produce known cum-loss curves. - A few prepaids per vintage to produce a known CPR. - Seeded RNG for full determinism. - Unit tests assert metric values with relative tolerance 1e-9 for stock metrics (cum loss, pool factor) and 1e-6 for annualized rate metrics (CPR/CDR) to absorb the  $1 - (1-x)^{12}$  float drift. - Explicit fixtures for: (a) a loan that cures after crossing the DPD threshold (verifies stickiness), (b) a default-month with nonzero principal\_payment (verifies prepay numerator exclusion), (c) a vintage that doesn't reach the max MOB (verifies n\_vintages reporting).

---

## 7. Deliverables & Sequence

1. vintage\_curves/ package skeleton + schema.py, tape.py. 2. mob.py, default\_policy.py, segment.py. 3. metrics.py (per-vintage curves) + tests. 4. aggregate.py (balance-weighted) + tests. 5. summary.py, rolls.py + tests. 6. output.py, plotting.py. 7. analyzer.py façade + end-to-end test. 8. examples/ — a notebook-style script using the synthetic tape. 9. Top-level README section for the package.

---

## 8. Open Items / Future Work

- Net-loss curves with recovery lag modeling.
- Multi-file & Parquet loaders.
- Polars backend.
- Streamlit dashboard.
- Stratification reports (segment × segment grids).
- Statistical curve fitting (logistic / Weibull) for forecasting.

## pyproject.toml

```
1 [build-system]
2 requires = ["hatchling"]
3 build-backend = "hatchling.build"
4
5 [project]
6 name = "vintage-curves"
7 version = "0.1.0"
8 description = "Vintage performance curves for consumer-loan tapes."
9 requires-python = ">=3.11"
10 dependencies = [
11     "pandas>=2.0",
12     "numpy>=1.24",
13     "pyyaml>=6.0",
14     "matplotlib>=3.7",
15     "openpyxl>=3.1",
16 ]
17
18 [project.optional-dependencies]
19 test = ["pytest>=7.0"]
20
21 [tool.hatch.build.targets.wheel]
22 packages = ["vintage_curves"]
23
24 [tool.pytest.ini_options]
25 testpaths = ["tests"]
```



## vintage\_curves/\_\_init\_\_.py

```
1  """Vintage performance curves for consumer-loan tapes."""
2
3  from vintage_curves.aggregate import aggregate_curves
4  from vintage_curves.analyzer import SegmentResult, VintageAnalyzer
5  from vintage_curves.default_policy import DefaultPolicy
6  from vintage_curves.metrics import VintageMetrics
7  from vintage_curves.mob import add_mob_and_vintage, find_mob_gaps
8  from vintage_curves.output import to_csv_dir, to_excel
9  from vintage_curves.plotting import plot_curves
10 from vintage_curves.rolls import assign_buckets, transition_matrix
11 from vintage_curves.schema import CanonicalColumns, SchemaMapper
12 from vintage_curves.segment import Segment, SegmentSet
13 from vintage_curves.summary import vintage_summary
14 from vintage_curves.tape import LoanTape
15
16 __all__ = [
17     "CanonicalColumns",
18     "DefaultPolicy",
19     "LoanTape",
20     "SchemaMapper",
21     "Segment",
22     "SegmentResult",
23     "SegmentSet",
24     "VintageAnalyzer",
25     "VintageMetrics",
26     "add_mob_and_vintage",
27     "aggregate_curves",
28     "assign_buckets",
29     "find_mob_gaps",
30     "plot_curves",
31     "to_csv_dir",
32     "to_excel",
33     "transition_matrix",
34     "vintage_summary",
35 ]
```

## vintage\_curves/schema.py

```
1  """Canonical column names, schema mapping, and tape validation."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from pathlib import Path
7  from typing import Mapping
8
9  import numpy as np
10 import pandas as pd
11 import yaml
12
13
14 class CanonicalColumns:
15     # Loan-level (constant per loan_id)
16     LOAN_ID = "loan_id"
17     ORIGINATION_DATE = "origination_date"
18     ORIGINAL_BALANCE = "original_balance"
19     ORIGINAL_TERM = "original_term"
20     APR = "apr"
21     PRODUCT_TYPE = "product_type"
22     DEFERRAL_PERIOD = "deferral_period"
23
24     # Borrower-level (constant per loan_id)
25     FICO = "fico"
26     DTI = "dti"
27     INCOME = "income"
28     STATE = "state"
29
30     # Performance (one row per loan-month)
31     AS_OF_DATE = "as_of_date"
32     OUTSTANDING_BALANCE = "outstanding_balance"
33     SCHEDULED_PRINCIPAL = "scheduled_principal"
34     PRINCIPAL_PAYMENT = "principal_payment"
35     INTEREST_PAYMENT = "interest_payment"
36     FEES_PAID = "fees_paid"
37     DAYS_PAST_DUE = "days_past_due"
38     CHARGE_OFF_DATE = "charge_off_date"
39     CHARGE_OFF_AMOUNT = "charge_off_amount"
40
41
42 LOAN_LEVEL_COLUMNS = frozenset({
43     CanonicalColumns.LOAN_ID,
44     CanonicalColumns.ORIGINATION_DATE,
45     CanonicalColumns.ORIGINAL_BALANCE,
46     CanonicalColumns.ORIGINAL_TERM,
47     CanonicalColumns.APR,
48     CanonicalColumns.PRODUCT_TYPE,
49     CanonicalColumns.DEFERRAL_PERIOD,
50     CanonicalColumns.FICO,
51     CanonicalColumns.DTI,
52     CanonicalColumns.INCOME,
53     CanonicalColumns.STATE,
54 })
55
56 PERF_LEVEL_COLUMNS = frozenset({
57     CanonicalColumns.AS_OF_DATE,
58     CanonicalColumns.OUTSTANDING_BALANCE,
59     CanonicalColumns.SCHEDULED_PRINCIPAL,
60     CanonicalColumns.PRINCIPAL_PAYMENT,
61     CanonicalColumns.INTEREST_PAYMENT,
62     CanonicalColumns.FEES_PAID,
63     CanonicalColumns.DAYS_PAST_DUE,
64     CanonicalColumns.CHARGE_OFF_DATE,
65     CanonicalColumns.CHARGE_OFF_AMOUNT,
66 })
67
68 REQUIRED_LOAN_COLUMNS = frozenset({
69     CanonicalColumns.LOAN_ID,
70     CanonicalColumns.ORIGINATION_DATE,
71     CanonicalColumns.ORIGINAL_BALANCE,
72     CanonicalColumns.ORIGINAL_TERM,
```

```

73     CanonicalColumns.APR,
74 })
75
76 REQUIRED_PERF_COLUMNS = frozenset({
77     CanonicalColumns.AS_OF_DATE,
78     CanonicalColumns.OUTSTANDING_BALANCE,
79     CanonicalColumns.SCHEDULED_PRINCIPAL,
80     CanonicalColumns.PRINCIPAL_PAYMENT,
81     CanonicalColumns.DAYS_PAST_DUE,
82 })
83
84 DATE_COLUMNS = frozenset({
85     CanonicalColumns.ORIGINATION_DATE,
86     CanonicalColumns.AS_OF_DATE,
87     CanonicalColumns.CHARGE_OFF_DATE,
88 })
89
90 FLOAT_COLUMNS = frozenset({
91     CanonicalColumns.ORIGINAL_BALANCE,
92     CanonicalColumns.APR,
93     CanonicalColumns.DTI,
94     CanonicalColumns.INCOME,
95     CanonicalColumns.OUTSTANDING_BALANCE,
96     CanonicalColumns.SCHEDULED_PRINCIPAL,
97     CanonicalColumns.PRINCIPAL_PAYMENT,
98     CanonicalColumns.INTEREST_PAYMENT,
99     CanonicalColumns.FEES_PAID,
100    CanonicalColumns.CHARGE_OFF_AMOUNT,
101 })
102
103 INT_COLUMNS = frozenset({
104     CanonicalColumns.ORIGINAL_TERM,
105     CanonicalColumns.DEFERRAL_PERIOD,
106     CanonicalColumns.FICO,
107     CanonicalColumns.DAYS_PAST_DUE,
108 })
109
110 STR_COLUMNS = frozenset({
111     CanonicalColumns.LOAN_ID,
112     CanonicalColumns.PRODUCT_TYPE,
113     CanonicalColumns.STATE,
114 })
115
116
117 class SchemaValidationError(ValueError):
118     """Raised when a tape fails canonical-schema validation."""
119
120
121 @dataclass
122 class SchemaMapper:
123     """Renames vendor columns to canonical names and coerces dtypes.
124
125     The mapping dict is keyed by canonical name and valued by the source
126     column name in the user's tape.
127     """
128
129     mapping: Mapping[str, str]
130
131     @classmethod
132     def from_yaml(cls, path: str | Path) -> "SchemaMapper":
133         with open(path) as f:
134             mapping = yaml.safe_load(f) or {}
135         if not isinstance(mapping, dict):
136             raise SchemaValidationError(
137                 f"Schema YAML at {path} must be a mapping of canonical->source names."
138             )
139         unknown = set(mapping) - LOAN_LEVEL_COLUMNS - PERF_LEVEL_COLUMNS
140         if unknown:
141             raise SchemaValidationError(
142                 f"Unknown canonical columns in schema: {sorted(unknown)}"
143             )
144         return cls(mapping=dict(mapping))
145
146     def rename(self, df: pd.DataFrame) -> pd.DataFrame:
147         """Return a copy of df with source columns renamed to canonical names.

```

```

148
149 Source columns not referenced in the mapping are dropped – the analyzer
150 only operates on the canonical schema.
151 """
152 # canonical -> source, invert to source -> canonical, keeping only sources present
153 present = {src: canon for canon, src in self.mapping.items() if src in df.columns}
154 missing_sources = [src for src in self.mapping.values() if src not in df.columns]
155 if missing_sources:
156     raise SchemaValidationError(
157         f"Source columns referenced in schema but absent from tape: {missing_sources}"
158     )
159 keep = list(present.keys())
160 return df[keep].rename(columns=present)
161
162 def validate_and_coerce(self, df: pd.DataFrame) -> pd.DataFrame:
163     """Validate required columns are present and non-empty; coerce dtypes."""
164     missing_required = (REQUIRED_LOAN_COLUMNS | REQUIRED_PERF_COLUMNS) - set(df.columns)
165     if missing_required:
166         raise SchemaValidationError(
167             f"Required canonical columns missing from tape after mapping: "
168             f"{sorted(missing_required)}"
169         )
170
171     out = df.copy()
172
173     for col in DATE_COLUMNS & set(out.columns):
174         out[col] = pd.to_datetime(out[col], errors="coerce")
175
176     for col in FLOAT_COLUMNS & set(out.columns):
177         out[col] = pd.to_numeric(out[col], errors="coerce").astype(float)
178
179     for col in INT_COLUMNS & set(out.columns):
180         # Nullable Int64 so optional ints (e.g. deferral_period) can carry NaN
181         out[col] = pd.to_numeric(out[col], errors="coerce").astype("Int64")
182
183     for col in STR_COLUMNS & set(out.columns):
184         out[col] = out[col].astype("string")
185
186     # All-NaN check on hard-required columns
187     for col in REQUIRED_LOAN_COLUMNS | REQUIRED_PERF_COLUMNS:
188         if col in out.columns and out[col].isna().all():
189             raise SchemaValidationError(
190                 f"Required column '{col}' is present but entirely NaN/empty."
191             )
192
193     return out

```

## vintage\_curves/tape.py

```
1  """LoanTape: load a CSV tape, apply schema, split into loan-level + perf frames."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from pathlib import Path
7
8  import pandas as pd
9
10 from vintage_curves.schema import (
11     LOAN_LEVEL_COLUMNS,
12     PERF_LEVEL_COLUMNS,
13     CanonicalColumns,
14     SchemaMapper,
15     SchemaValidationError,
16 )
17
18
19 @dataclass
20 class LoanTape:
21     """Canonicalized tape split into loan-level and perf-level frames.
22
23     `loans`: one row per `loan_id` with loan + borrower characteristics.
24     `perf`: one row per (loan_id, as_of_date), sorted.
25     """
26
27     loans: pd.DataFrame
28     perf: pd.DataFrame
29
30     @classmethod
31     def from_csv(
32         cls,
33         path: str | Path,
34         schema: str | Path | SchemaMapper,
35         **read_csv_kwargs,
36     ) -> "LoanTape":
37         mapper = schema if isinstance(schema, SchemaMapper) else SchemaMapper.from_yaml(schema)
38         raw = pd.read_csv(path, **read_csv_kwargs)
39         return cls.from_dataframe(raw, mapper)
40
41     @classmethod
42     def from_dataframe(cls, df: pd.DataFrame, mapper: SchemaMapper) -> "LoanTape":
43         renamed = mapper.rename(df)
44         canonical = mapper.validate_and_coerce(renamed)
45         loans, perf = _split(canonical)
46         _validate_loan_constancy(canonical, loans)
47         perf = perf.sort_values(
48             [CanonicalColumns.LOAN_ID, CanonicalColumns.AS_OF_DATE],
49             kind="mergesort",
50         ).reset_index(drop=True)
51         return cls(loans=loans, perf=perf)
52
53     @property
54     def n_loans(self) -> int:
55         return len(self.loans)
56
57     @property
58     def n_loan_months(self) -> int:
59         return len(self.perf)
60
61
62 def _split(canonical: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
63     loan_cols = [c for c in canonical.columns if c in LOAN_LEVEL_COLUMNS]
64     perf_cols = [c for c in canonical.columns if c in PERF_LEVEL_COLUMNS]
65     if CanonicalColumns.LOAN_ID not in loan_cols:
66         raise SchemaValidationError("loan_id column missing after canonicalization.")
67
68     loans = (
69         canonical[loan_cols]
70         .drop_duplicates(subset=[CanonicalColumns.LOAN_ID])
71         .reset_index(drop=True)
72     )
```

```

73     perf = canonical[[CanonicalColumns.LOAN_ID, *perf_cols]].copy()
74     return loans, perf
75
76
77 def _validate_loan_constancy(canonical: pd.DataFrame, loans: pd.DataFrame) -> None:
78     """Confirm loan-level columns don't vary within a loan_id."""
79     loan_id = CanonicalColumns.LOAN_ID
80     cols_to_check = [c for c in loans.columns if c != loan_id]
81     if not cols_to_check:
82         return
83     # nunique across rows of the same loan_id; >1 means inconsistent
84     nun = canonical.groupby(loan_id)[cols_to_check].nunique(dropna=False)
85     bad = nun[(nun > 1).any(axis=1)]
86     if not bad.empty:
87         offenders = bad.index.tolist()[:5]
88         raise SchemaValidationError(
89             f"Loan-level columns vary within loan_id for {len(bad)} loan(s); "
90             f"first offenders: {offenders}"
91         )

```

## vintage\_curves/mob.py

```
1  """MOB, vintage, and BOP-balance derivation on a perf frame."""
2
3  from __future__ import annotations
4
5  from typing import Literal
6
7  import pandas as pd
8
9  from vintage_curves.schema import CanonicalColumns
10
11  VintageFreq = Literal["M", "Q", "A"]
12
13  # Column names added by this module
14  MOB = "mob"
15  VINTAGE = "vintage"
16  BOP_BALANCE = "bop_balance"
17
18
19  def add_mob_and_vintage(
20      perf: pd.DataFrame,
21      loans: pd.DataFrame,
22      vintage_freq: VintageFreq = "Q",
23  ) -> pd.DataFrame:
24      """Return a copy of `perf` with `mob`, `vintage`, and `bop_balance` added.
25
26      Conventions
27      -----
28      - MOB 1 is the first reporting period of the loan (origination month-end).
29        Computed as `(as_of.to_period('M') - origination.to_period('M')).n + 1`.
30      - `vintage` is `origination_date.to_period(vintage_freq)` (calendar buckets,
31        not fiscal). Accepts 'M', 'Q', or 'A'.
32      - `bop_balance` is the prior MOB's `outstanding_balance` within each loan_id,
33        with `original_balance` substituted at MOB 1. NaN if the prior MOB is
34        missing from the tape (gap).
35      """
36      if vintage_freq not in ("M", "Q", "A", "Y"):
37          raise ValueError(f"vintage_freq must be one of 'M','Q','A','Y', got {vintage_freq!r}")
38      pd_freq = "Y" if vintage_freq == "A" else vintage_freq
39
40      loan_id = CanonicalColumns.LOAN_ID
41      orig_dt = CanonicalColumns.ORIGINATION_DATE
42      as_of = CanonicalColumns.AS_OF_DATE
43      orig_bal = CanonicalColumns.ORIGINAL_BALANCE
44      eop_bal = CanonicalColumns.OUTSTANDING_BALANCE
45
46      needed = loans[[loan_id, orig_dt, orig_bal]]
47      out = perf.merge(needed, on=loan_id, how="left", validate="many_to_one")
48
49      as_of_period = out[as_of].dt.to_period("M")
50      orig_period = out[orig_dt].dt.to_period("M")
51      out[MOB] = (as_of_period.astype("int64") - orig_period.astype("int64")) + 1
52
53      out[VINTAGE] = out[orig_dt].dt.to_period(pd_freq)
54
55      out = out.sort_values([loan_id, MOB], kind="mergesort").reset_index(drop=True)
56
57      prior_eop = out.groupby(loan_id, sort=False)[eop_bal].shift(1)
58      out[BOP_BALANCE] = prior_eop.where(out[MOB] > 1, out[orig_bal])
59
60      return out.drop(columns=[orig_dt, orig_bal])
61
62
63  def find_mob_gaps(perf_with_mob: pd.DataFrame) -> pd.DataFrame:
64      """Return rows where a loan's MOB sequence skips a month (diagnostic only)."""
65      loan_id = CanonicalColumns.LOAN_ID
66      diffs = perf_with_mob.groupby(loan_id, sort=False)[MOB].diff()
67      gap_mask = (diffs > 1).fillna(False)
68      return perf_with_mob.loc[gap_mask, [loan_id, MOB]].assign(
69          gap_size=diffs[gap_mask].astype(int)
70      )
```

## vintage\_curves/default\_policy.py

```
1  """Default-event detection per the plan's two policies (sticky)."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from typing import Literal
7
8  import numpy as np
9  import pandas as pd
10
11  from vintage_curves.mob import MOB
12  from vintage_curves.schema import CanonicalColumns
13
14  # Columns added by `apply()`
15  IS_DEFAULTED = "is_defaulted"      # sticky: True from default month onward
16  DEFAULT_EVENT = "default_event"    # True only in the month of default
17  DEFAULT_LOSS = "default_loss_amount" # populated only on the default event row
18
19  PolicyKind = Literal["charge_off_field", "dpd_threshold"]
20  OnConflict = Literal["raise", "charge_off_wins", "dpd_wins"]
21
22
23  class DefaultPolicyError(ValueError):
24      """Raised when policy inputs conflict or are inapplicable."""
25
26
27  @dataclass(frozen=True)
28  class DefaultPolicy:
29      kind: PolicyKind
30      dpd_threshold_value: int | None = None
31      on_conflict: OnConflict = "raise"
32
33      @classmethod
34      def charge_off_field(cls) -> "DefaultPolicy":
35          return cls(kind="charge_off_field")
36
37      @classmethod
38      def dpd_threshold(cls, n: int, on_conflict: OnConflict = "raise") -> "DefaultPolicy":
39          if n <= 0:
40              raise ValueError("dpd_threshold must be positive.")
41          return cls(kind="dpd_threshold", dpd_threshold_value=n, on_conflict=on_conflict)
42
43  # ----- apply
44
45  def apply(self, perf_with_mob: pd.DataFrame) -> pd.DataFrame:
46      """Return a copy of `perf_with_mob` with default flags + loss amount.
47
48      Adds columns `is_defaulted` (sticky bool), `default_event` (bool;
49      True only on the first default month), and `default_loss_amount`
50      (float; 0 except on the default event row).
51      """
52      if self.kind == "charge_off_field":
53          return self._apply_charge_off(perf_with_mob)
54      if self.kind == "dpd_threshold":
55          return self._apply_dpd(perf_with_mob)
56      raise DefaultPolicyError(f"Unknown policy kind: {self.kind!r}")
57
58  # ----- charge-off
59
60  def _apply_charge_off(self, perf: pd.DataFrame) -> pd.DataFrame:
61      co_date = CanonicalColumns.CHARGE_OFF_DATE
62      co_amt = CanonicalColumns.CHARGE_OFF_AMOUNT
63      loan_id = CanonicalColumns.LOAN_ID
64
65      if co_date not in perf.columns:
66          raise DefaultPolicyError(
67              "charge_off_field policy requires `charge_off_date` in the tape."
68          )
69
70      out = perf.copy()
71      out = out.sort_values([loan_id, MOB], kind="mergesort").reset_index(drop=True)
72
```



```

73     event = out[co_date].notna()
74     # Stickiness: mark only the first CO row per loan (guards against any
75     # malformed tape with multiple CO rows for the same loan).
76     out[DEFAULT_EVENT] = _mark_first_true_per_group(event, out[loan_id])
77     out[IS_DEFAULTED] = out.groupby(loan_id, sort=False)[DEFAULT_EVENT].cummax().astype(bool)
78
79     loss = pd.Series(0.0, index=out.index)
80     if co_amt in out.columns:
81         loss = out[co_amt].fillna(0.0).where(out[DEFAULT_EVENT], 0.0)
82     out[DEFAULT_LOSS] = loss
83     return out
84
85 # ----- DPD policy
86
87 def _apply_dpd(self, perf: pd.DataFrame) -> pd.DataFrame:
88     loan_id = CanonicalColumns.LOAN_ID
89     dpd = CanonicalColumns.DAYS_PAST_DUE
90     co_date = CanonicalColumns.CHARGE_OFF_DATE
91     eop = CanonicalColumns.OUTSTANDING_BALANCE
92     n = self.dpd_threshold_value
93     assert n is not None
94
95     if dpd not in perf.columns:
96         raise DefaultPolicyError(
97             "dpd_threshold policy requires `days_past_due` in the tape."
98         )
99
100    out = perf.copy()
101    out = out.sort_values([loan_id, MOB], kind="mergesort").reset_index(drop=True)
102
103    # Conflict detection: tape also populates charge_off_date
104    if co_date in out.columns and out[co_date].notna().any():
105        if self.on_conflict == "raise":
106            offenders = (
107                out.loc[out[co_date].notna(), loan_id].drop_duplicates().head(5).tolist()
108            )
109            raise DefaultPolicyError(
110                f"dpd_threshold policy chosen but tape also populates "
111                f"charge_off_date for some loans (e.g. {offenders}). "
112                f"Pass DefaultPolicy.dpd_threshold(n, on_conflict='charge_off_wins') "
113                f"or 'dpd_wins' to resolve."
114            )
115
116    dpd_event = (out[dpd].fillna(0).astype("int64") >= n)
117
118    if co_date in out.columns and self.on_conflict == "charge_off_wins":
119        # For loans where charge_off_date is populated, use that as the event;
120        # for others, use DPD threshold.
121        co_loans = set(out.loc[out[co_date].notna(), loan_id].unique())
122        co_event = out[co_date].notna()
123        use_co = out[loan_id].isin(co_loans)
124        event = np.where(use_co, co_event, dpd_event)
125        out["__event_raw"] = event
126        event_series = pd.Series(event, index=out.index)
127    else:
128        # dpd_wins (or no conflict): ignore charge_off_date, use DPD purely
129        event_series = dpd_event
130
131    out[DEFAULT_EVENT] = _mark_first_true_per_group(event_series, out[loan_id])
132    out[IS_DEFAULTED] = out.groupby(loan_id, sort=False)[DEFAULT_EVENT].cummax().astype(bool)
133
134    # Loss = EOP outstanding_balance at the default month (gross), unless
135    # we're in charge_off_wins mode for a CO loan – then use charge_off_amount.
136    loss = out[eop].fillna(0.0).where(out[DEFAULT_EVENT], 0.0)
137    co_amt = CanonicalColumns.CHARGE_OFF_AMOUNT
138    if (
139        co_amt in out.columns
140        and co_date in out.columns
141        and self.on_conflict == "charge_off_wins"
142    ):
143        co_loans = set(out.loc[out[co_date].notna(), loan_id].unique())
144        on_co_event = out[DEFAULT_EVENT] & out[loan_id].isin(co_loans) & out[co_date].notna()
145        loss = loss.where(~on_co_event, out[co_amt].fillna(0.0))
146    out[DEFAULT_LOSS] = loss
147

```

```
148         if "__event_raw" in out.columns:
149             out = out.drop(columns="__event_raw")
150         return out
151
152
153 def _mark_first_true_per_group(flag: pd.Series, group: pd.Series) -> pd.Series:
154     """For each group, return a bool series that is True only at the first
155     True position within that group (and False elsewhere, including after)."""
156     flag = flag.astype(bool)
157     cum = flag.groupby(group, sort=False).cumsum()
158     return (flag & (cum == 1))
159
160
```

## vintage\_curves/metrics.py

```
1  """Per-vintage performance curves.
2
3  Each public method returns a wide DataFrame indexed by `vintage`, columned by
4  `mob`. Cells beyond a vintage's max observable MOB are NaN.
5
6  Metric definitions follow §4 of the plan:
7  - Performing-at-BOP = loans that were not yet defaulted at the start of the
8    period. A loan defaulting this period is performing-at-BOP but not
9    strictly-performing.
10 - Strictly-performing = not defaulted as of EOP (excludes the default-event
11   month and all later months).
12 - SMM denominator excludes both scheduled principal and the BOP balance of
13   loans defaulting this period (PSA convention).
14 - Prepay numerator counts only strictly-performing loan-months so charge-off
15   write-downs cannot leak into prepays.
16 """
17
18 from __future__ import annotations
19
20 from dataclasses import dataclass
21 from functools import cached_property
22
23 import numpy as np
24 import pandas as pd
25
26 from vintage_curves.default_policy import (
27     DEFAULT_EVENT,
28     DEFAULT_LOSS,
29     IS_DEFAULTED,
30     DefaultPolicy,
31 )
32 from vintage_curves.mob import BOP_BALANCE, MOB, VINTAGE, VintageFreq, add_mob_and_vintage
33 from vintage_curves.schema import CanonicalColumns
34 from vintage_curves.segment import Segment
35
36
37 @dataclass
38 class VintageMetrics:
39     """Per-vintage curves, computed once and cached.
40
41     Construct via `VintageMetrics.from_segment(seg, default_policy, vintage_freq=...)\`.
42     """
43
44     perf_flagged: pd.DataFrame # one row per loan-month; has mob, vintage, default flags, bop_balance
45     loans: pd.DataFrame # one row per loan_id; carries original_balance, apr, etc.
46
47     @classmethod
48     def from_segment(
49         cls,
50         segment: Segment,
51         default_policy: DefaultPolicy,
52         vintage_freq: VintageFreq = "Q",
53     ) -> "VintageMetrics":
54         perf = add_mob_and_vintage(segment.perf, segment.loans, vintage_freq=vintage_freq)
55         flagged = default_policy.apply(perf)
56         return cls(perf_flagged=flagged, loans=segment.loans)
57
58     # ----- per-vintage scalars
59
60     @cached_property
61     def vintage_original_balance(self) -> pd.Series:
62         """OB_v: sum of original_balance per vintage (over segment loans)."""
63         loan_id = CanonicalColumns.LOAN_ID
64         orig_bal = CanonicalColumns.ORIGINAL_BALANCE
65         # Pull vintage from the flagged perf (already computed per loan-month);
66         # take the first occurrence per loan to attach vintage to each loan.
67         vintage_per_loan = (
68             self.perf_flagged[[loan_id, VINTAGE]].drop_duplicates(subset=[loan_id])
69         )
70         merged = self.loans.merge(vintage_per_loan, on=loan_id, how="inner")
71         return merged.groupby(VINTAGE, observed=True)[orig_bal].sum().sort_index()
72
```

```

73 # ----- per-(v, mob) aggs
74
75 @cached_property
76 def aggregates(self) -> pd.DataFrame:
77     """Raw per-(vintage, mob) sums used to build every curve.
78
79     Columns:
80     - bop_bal_perf:  $\Sigma$  bop_balance over loans performing at BOP
81     - scheduled:  $\Sigma$  scheduled_principal over strictly-performing rows
82     - unscheduled:  $\Sigma$  max(principal_payment - scheduled_principal, 0) over strictly-performing rows
83     - default_bal:  $\Sigma$  bop_balance over rows where default_event = True
84     - loss_amount:  $\Sigma$  default_loss_amount
85     - eop_bal_all:  $\Sigma$  outstanding_balance over all rows (for pool factor)
86     - apr_x_bop_perf:  $\Sigma$  apr * bop_balance over loans performing at BOP (for WA coupon)
87     - n_loans: count of loan-months (diagnostic)
88     """
89     loan_id = CanonicalColumns.LOAN_ID
90     eop = CanonicalColumns.OUTSTANDING_BALANCE
91     sched = CanonicalColumns.SCHEDULED_PRINCIPAL
92     prin_pay = CanonicalColumns.PRINCIPAL_PAYMENT
93     apr_col = CanonicalColumns.APR
94
95     f = self.perf_flagged
96     if apr_col not in f.columns:
97         f = f.merge(self.loans[[loan_id, apr_col]], on=loan_id, how="left", validate="many_to_one")
98
99     strictly_perf = ~f[IS_DEFAULTED]
100    bop_perf = strictly_perf | f[DEFAULT_EVENT] # performing at BOP
101    de = f[DEFAULT_EVENT]
102
103    sched_clean = f[sched].fillna(0.0)
104    prin_clean = f[prin_pay].fillna(0.0)
105    bop_clean = f[BOP_BALANCE].fillna(0.0)
106
107    work = pd.DataFrame({
108        VINTAGE: f[VINTAGE],
109        MOB: f[MOB],
110        "bop_bal_perf": np.where(bop_perf, bop_clean, 0.0),
111        "scheduled": np.where(strictly_perf, sched_clean, 0.0),
112        "unscheduled": np.where(strictly_perf, np.clip(prin_clean - sched_clean, 0.0, None), 0.0),
113        "default_bal": np.where(de, bop_clean, 0.0),
114        "loss_amount": f[DEFAULT_LOSS].fillna(0.0).to_numpy(),
115        "eop_bal_all": f[eop].fillna(0.0).to_numpy(),
116        "apr_x_bop_perf": np.where(bop_perf, f[apr_col].astype(float).fillna(0.0) * bop_clean, 0.0),
117        "n_loans": 1,
118    })
119    agg = work.groupby([VINTAGE, MOB], observed=True, sort=True).sum()
120    return agg
121
122 # ----- curves
123
124 def cum_gross_loss(self) -> pd.DataFrame:
125     """Cumulative gross loss % of original balance, per vintage x MOB."""
126     loss_wide = self._pivot("loss_amount")
127     cum = loss_wide.cumsum(axis=1)
128     return cum.div(self.vintage_original_balance, axis=0)
129
130 def cum_prepay(self) -> pd.DataFrame:
131     """Cumulative voluntary prepay % of original balance, per vintage x MOB."""
132     unsched_wide = self._pivot("unscheduled")
133     cum = unsched_wide.cumsum(axis=1)
134     return cum.div(self.vintage_original_balance, axis=0)
135
136 def pool_factor(self) -> pd.DataFrame:
137     """EOP balance / original balance, per vintage x MOB."""
138     eop_wide = self._pivot("eop_bal_all")
139     return eop_wide.div(self.vintage_original_balance, axis=0)
140
141 def smm(self) -> pd.DataFrame:
142     """Single Monthly Mortality, per vintage x MOB.
143
144     Numerator: unscheduled principal from strictly-performing loan-months.
145     Denominator: BOP balance of performing loans - scheduled principal of
146     strictly-performing loans - BOP balance of loans defaulting this period.
147     NaN where denominator  $\leq 0$ .

```

```

148         """
149         unsched = self._pivot("unscheduled")
150         bop = self._pivot("bop_bal_perf")
151         sched = self._pivot("scheduled")
152         dbal = self._pivot("default_bal")
153         denom = bop - sched - dbal
154         out = unsched.where(denom > 0).div(denom.where(denom > 0))
155         return out
156
157     def cpr(self) -> pd.DataFrame:
158         """Annualized CPR = 1 - (1 - SMM)^12."""
159         smm = self.smm()
160         return 1 - (1 - smm) ** 12
161
162     def mdr(self) -> pd.DataFrame:
163         """Monthly default rate: default_bal / BOP performing balance.
164
165         NaN where denominator ≤ 0.
166         """
167         dbal = self._pivot("default_bal")
168         bop = self._pivot("bop_bal_perf")
169         return dbal.where(bop > 0).div(bop.where(bop > 0))
170
171     def cdr(self) -> pd.DataFrame:
172         """Annualized CDR = 1 - (1 - MDR)^12."""
173         mdr = self.mdr()
174         return 1 - (1 - mdr) ** 12
175
176     def wa_coupon(self) -> pd.DataFrame:
177         """Balance-weighted APR over performing-at-BOP loans, per vintage × MOB."""
178         apr_bal = self._pivot("apr_x_bop_perf")
179         bop = self._pivot("bop_bal_perf")
180         return apr_bal.where(bop > 0).div(bop.where(bop > 0))
181
182     # ----- helpers
183
184     def _pivot(self, column: str) -> pd.DataFrame:
185         """Return a wide vintage × MOB frame for an aggregate column."""
186         return self.aggregates[column].unstack(MOB)
187
188     def available_metrics(self) -> list[str]:
189         return [
190             "cum_gross_loss", "cum_prepay", "pool_factor",
191             "smm", "cpr", "mdr", "cdr", "wa_coupon",
192         ]
193
194     def curve(self, name: str) -> pd.DataFrame:
195         """Dispatch by metric name."""
196         if name not in self.available_metrics():
197             raise ValueError(f"Unknown metric {name!r}; available: {self.available_metrics()}")
198         return getattr(self, name)()

```

## vintage\_curves/segment.py

```
1  """Segment: a labeled filter over a LoanTape using a pandas query string."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from typing import Iterable
7
8  import pandas as pd
9
10 from vintage_curves.schema import CanonicalColumns
11 from vintage_curves.tape import LoanTape
12
13
14 @dataclass
15 class Segment:
16     """A subset of a LoanTape selected by a pandas-`query()` string on loan-level attrs.
17
18     The filter operates on `tape.loans` (one row per loan_id); the matching
19     perf rows are derived by `loan_id` membership. Time-varying attributes
20     (e.g. days_past_due) are deliberately not queryable here – segmentation
21     is a loan-cohort concept.
22     """
23
24     label: str
25     loans: pd.DataFrame
26     perf: pd.DataFrame
27     query: str | None = None
28
29     @classmethod
30     def from_tape(
31         cls,
32         tape: LoanTape,
33         query: str | None,
34         label: str,
35     ) -> "Segment":
36         if query is None or query.strip() == "":
37             loans = tape.loans
38         else:
39             try:
40                 loans = tape.loans.query(query)
41             except Exception as e: # noqa: BLE001 – re-raise with context
42                 raise SegmentQueryError(
43                     f"Failed to evaluate segment query {query!r}: {e}"
44                 ) from e
45             loan_ids = loans[CanonicalColumns.LOAN_ID]
46             perf = tape.perf[tape.perf[CanonicalColumns.LOAN_ID].isin(loan_ids)].copy()
47             return cls(label=label, loans=loans.copy(), perf=perf, query=query)
48
49     @property
50     def n_loans(self) -> int:
51         return len(self.loans)
52
53     @property
54     def n_loan_months(self) -> int:
55         return len(self.perf)
56
57
58 class SegmentQueryError(ValueError):
59     """Raised when a segment query string cannot be evaluated against loan-level attrs."""
60
61
62 @dataclass
63 class SegmentSet:
64     """A collection of named segments for batch metric production."""
65
66     segments: list[Segment]
67
68     @classmethod
69     def from_tape(
70         cls,
71         tape: LoanTape,
72         specs: Iterable[tuple[str, str]],
```

```
73     ) -> "SegmentSet":
74         """`specs` is an iterable of (label, query) pairs."""
75         return cls(segments=[Segment.from_tape(tape, q, label=lbl) for lbl, q in specs])
76
77     def __iter__(self):
78         return iter(self.segments)
79
80     def __len__(self) -> int:
81         return len(self.segments)
```

## vintage\_curves/aggregate.py

```
1  """Aggregate curves across vintages.
2
3  Each call returns a DataFrame indexed by MOB with one column per requested
4  metric plus `n_vintages` (count of vintages observable at that MOB).
5
6  Weighting schemes (see §4 of the plan):
7  - "original": weight stock metrics by OB_v; flow metrics aggregate raw $ flows.
8  - "beginning": weight stock metrics by BOP balance at MOB t; flow metrics
9                  aggregate raw $ flows (so flow metrics coincide with "original").
10 - "equal":      simple unweighted average across observable vintages.
11 """
12
13 from __future__ import annotations
14
15 from typing import Iterable, Literal
16
17 import numpy as np
18 import pandas as pd
19
20 from vintage_curves.metrics import VintageMetrics
21
22 Weighting = Literal["original", "beginning", "equal"]
23
24 STOCK_METRICS = frozenset({"cum_gross_loss", "cum_prepay", "pool_factor", "wa_coupon"})
25 FLOW_METRICS = frozenset({"smm", "cpr", "mdr", "cdr"})
26
27
28 def aggregate_curves(
29     metrics: VintageMetrics,
30     names: Iterable[str],
31     weighting: Weighting = "original",
32 ) -> pd.DataFrame:
33     """Aggregate per-vintage curves into a single MOB-indexed series per metric."""
34     names = list(names)
35     if weighting not in ("original", "beginning", "equal"):
36         raise ValueError(
37             f"weighting must be 'original', 'beginning', or 'equal'; got {weighting!r}"
38         )
39
40     pool_factor = metrics.pool_factor()
41     observable = pool_factor.notna()
42
43     weights = _build_weights(metrics, observable, weighting)
44
45     out: dict[str, pd.Series] = {}
46     for name in names:
47         if name in STOCK_METRICS:
48             out[name] = _aggregate_stock(getattr(metrics, name)(), weights, weighting)
49         elif name in FLOW_METRICS:
50             out[name] = _aggregate_flow(metrics, name, weighting)
51         else:
52             raise ValueError(
53                 f"Unknown metric {name!r}. "
54                 f"Stock: {sorted(STOCK_METRICS)}; flow: {sorted(FLOW_METRICS)}."
55             )
56
57     n_vintages = observable.sum(axis=0).astype("int64")
58     return pd.DataFrame(out).assign(n_vintages=n_vintages)
59
60
61 # ----- helpers
62
63
64 def _build_weights(
65     metrics: VintageMetrics,
66     observable: pd.DataFrame,
67     weighting: Weighting,
68 ) -> pd.DataFrame:
69     """Wide vintage × MOB weight matrix, NaN where unobservable."""
70     if weighting == "original":
71         ob = metrics.vintage_original_balance.reindex(observable.index)
72         w = pd.DataFrame(
```



```

73         np.where(observable.values, ob.to_numpy()[ :, None], np.nan),
74         index=observable.index,
75         columns=observable.columns,
76     )
77     return w
78     if weighting == "beginning":
79         bop = metrics._pivot("bop_bal_perf").reindex_like(observable)
80         return bop.where(observable)
81     # equal
82     return observable.astype(float).where(observable)
83
84
85 def _aggregate_stock(
86     per_vintage: pd.DataFrame,
87     weights: pd.DataFrame,
88     weighting: Weighting,
89 ) -> pd.Series:
90     """Weighted mean across vintages, skipping NaN cells."""
91     if weighting == "equal":
92         return per_vintage.mean(axis=0, skipna=True)
93     valid = per_vintage.notna() & weights.notna()
94     w_eff = weights.where(valid, 0.0)
95     num = (per_vintage.where(valid, 0.0) * w_eff).sum(axis=0, min_count=1)
96     denom = w_eff.sum(axis=0, min_count=1)
97     return num.where(denom > 0) / denom.where(denom > 0)
98
99
100 def _aggregate_flow(
101     metrics: VintageMetrics,
102     name: str,
103     weighting: Weighting,
104 ) -> pd.Series:
105     """Flow/rate metrics: $num / $denom across vintages, optionally annualized."""
106     if weighting == "equal":
107         per_v = getattr(metrics, name)()
108         return per_v.mean(axis=0, skipna=True)
109
110     if name in ("smm", "cpr"):
111         num = metrics._pivot("unscheduled")
112         denom = (
113             metrics._pivot("bop_bal_perf")
114             - metrics._pivot("scheduled")
115             - metrics._pivot("default_bal")
116         )
117         annualize = name == "cpr"
118     else: # mdr / cdr
119         num = metrics._pivot("default_bal")
120         denom = metrics._pivot("bop_bal_perf")
121         annualize = name == "cdr"
122
123     # Mask non-positive denominators per vintage before summing
124     valid = denom > 0
125     num_agg = num.where(valid, 0.0).sum(axis=0, min_count=1)
126     denom_agg = denom.where(valid, 0.0).sum(axis=0, min_count=1)
127     rate = num_agg.where(denom_agg > 0) / denom_agg.where(denom_agg > 0)
128     if annualize:
129         rate = 1 - (1 - rate) ** 12
130     return rate

```

## vintage\_curves/rolls.py

```
1  """Delinquency bucket assignment and roll-rate transition matrices."""
2
3  from __future__ import annotations
4
5  from typing import Literal, Sequence
6
7  import numpy as np
8  import pandas as pd
9
10 from vintage_curves.default_policy import IS_DEFAULTED
11 from vintage_curves.mob import MOB
12 from vintage_curves.schema import CanonicalColumns
13
14 BUCKET = "delinquency_bucket"
15 CO_BUCKET = "CO"
16
17 # Default bucket boundaries (per the refined plan): Current = 0-29 DPD.
18 DEFAULT_BOUNDARIES: tuple[tuple[str, int, int | None], ...] = (
19     ("Current", 0, 30),
20     ("30-59", 30, 60),
21     ("60-89", 60, 90),
22     ("90+", 90, None),
23 )
24
25
26 WeightMode = Literal["count", "balance"]
27
28
29 def assign_buckets(
30     perf_flagged: pd.DataFrame,
31     boundaries: Sequence[tuple[str, int, int | None]] = DEFAULT_BOUNDARIES,
32 ) -> pd.Series:
33     """Return a categorical Series of bucket labels per row.
34
35     Defaulted rows (sticky) are labeled 'CO' regardless of DPD. Non-defaulted
36     rows are bucketed by DPD using half-open intervals [lo, hi).
37     """
38     dpd = CanonicalColumns.DAYS_PAST_DUE
39     labels = [b[0] for b in boundaries] + [CO_BUCKET]
40     if len(set(labels)) != len(labels):
41         raise ValueError(f"Bucket labels must be unique; got {labels}")
42
43     dpd_vals = perf_flagged[dpd].fillna(0).astype("int64").to_numpy()
44     result = np.full(len(perf_flagged), "", dtype=object)
45     for label, lo, hi in boundaries:
46         if hi is None:
47             mask = dpd_vals >= lo
48         else:
49             mask = (dpd_vals >= lo) & (dpd_vals < hi)
50         result[mask] = label
51
52     # Override with CO for defaulted rows (sticky)
53     if IS_DEFAULTED in perf_flagged.columns:
54         is_co = perf_flagged[IS_DEFAULTED].to_numpy()
55         result[is_co] = CO_BUCKET
56
57     return pd.Series(
58         pd.Categorical(result, categories=labels, ordered=True),
59         index=perf_flagged.index,
60         name=BUCKET,
61     )
62
63
64 def transition_matrix(
65     perf_flagged: pd.DataFrame,
66     weight: WeightMode = "count",
67     per_mob: bool = False,
68     boundaries: Sequence[tuple[str, int, int | None]] = DEFAULT_BOUNDARIES,
69 ) -> pd.DataFrame:
70     """Roll-rate matrix from bucket(t) -> bucket(t+1) within each loan_id.
71
72     - weight="count": each transition contributes 1.
```

```

73 - weight="balance": each transition contributes the EOP balance at MOB t.
74 - per_mob=False: one pooled matrix indexed by from_bucket, columned by to_bucket.
75 - per_mob=True: MultiIndex (mob, from_bucket); rows normalize to 1 within (mob, from_bucket).
76 """
77 loan_id = CanonicalColumns.LOAN_ID
78 eop = CanonicalColumns.OUTSTANDING_BALANCE
79 labels = [b[0] for b in boundaries] + [CO_BUCKET]
80
81 f = perf_flagged.sort_values([loan_id, MOB], kind="mergesort").reset_index(drop=True)
82 f = f.assign(**{BUCKET: assign_buckets(f, boundaries)})
83 f["__next_bucket"] = f.groupby(loan_id, sort=False)[BUCKET].shift(-1)
84 f["__next_mob"] = f.groupby(loan_id, sort=False)[MOB].shift(-1)
85
86 # Keep only adjacent transitions (next observation is mob+1 in the tape)
87 transitions = f[(f["__next_mob"] - f[MOB]) == 1].copy()
88 if transitions.empty:
89     idx = pd.CategoricalIndex(labels, categories=labels, ordered=True, name=BUCKET)
90     return pd.DataFrame(0.0, index=idx, columns=idx)
91
92 if weight == "count":
93     transitions["__w"] = 1.0
94 elif weight == "balance":
95     transitions["__w"] = transitions[eop].fillna(0.0).astype(float)
96 else:
97     raise ValueError(f"weight must be 'count' or 'balance', got {weight!r}")
98
99 # Force both bucket columns to the same ordered Categorical so crosstab
100 # emits a full label x label matrix (zeros where no flow occurred).
101 cat = pd.CategoricalDtype(categories=labels, ordered=True)
102 transitions[BUCKET] = transitions[BUCKET].astype(cat)
103 transitions["__next_bucket"] = transitions["__next_bucket"].astype(cat)
104
105 if per_mob:
106     wide = pd.crosstab(
107         index=[transitions[MOB], transitions[BUCKET]],
108         columns=transitions["__next_bucket"],
109         values=transitions["__w"],
110         aggfunc="sum",
111         dropna=False,
112     ).fillna(0.0)
113     # Ensure every (observed mob, bucket) combination is present
114     mobs = sorted(transitions[MOB].unique())
115     full_index = pd.MultiIndex.from_product([mobs, labels], names=[MOB, BUCKET])
116     wide = wide.reindex(index=full_index, columns=labels, fill_value=0.0)
117 else:
118     wide = pd.crosstab(
119         index=transitions[BUCKET],
120         columns=transitions["__next_bucket"],
121         values=transitions["__w"],
122         aggfunc="sum",
123         dropna=False,
124     ).fillna(0.0)
125     wide = wide.reindex(index=labels, columns=labels, fill_value=0.0)
126
127 wide.columns.name = BUCKET + "__next"
128
129 # Normalize rows: each from-bucket sums to 1 (or 0 if no flows).
130 # Force plain (non-categorical) index to avoid categorical-alignment quirks.
131 wide.index = wide.index if per_mob else pd.Index(list(wide.index), name=wide.index.name)
132 wide.columns = pd.Index(list(wide.columns), name=wide.columns.name)
133 row_sums = wide.sum(axis=1)
134 denom = row_sums.where(row_sums > 0)
135 normalized = wide.divide(denom, axis="index").fillna(0.0)
136 return normalized

```

## vintage\_curves/summary.py

```
1  """Per-vintage summary scalars: WA Coupon, latest pool factor, WAL (realized + scheduled)."""
2
3  from __future__ import annotations
4
5  import numpy as np
6  import pandas as pd
7
8  from vintage_curves.metrics import VintageMetrics
9  from vintage_curves.mob import MOB, VINTAGE
10 from vintage_curves.schema import CanonicalColumns
11
12
13 def vintage_summary(metrics: VintageMetrics) -> pd.DataFrame:
14     """Return one row per vintage with summary scalars.
15
16     Columns:
17     - original_count:    number of loans in the vintage
18     - original_balance:  sum of original_balance (OB_v)
19     - wa_coupon:         OB-weighted average APR at origination
20     - pool_factor_latest: pool factor at the last observable MOB for the vintage
21     - wal_realized:      weighted average life over realized principal removals (months)
22     - wal_scheduled:    weighted average life of the implied level-pay schedule (months)
23     """
24     loan_id = CanonicalColumns.LOAN_ID
25     orig_bal = CanonicalColumns.ORIGINAL_BALANCE
26     apr_col = CanonicalColumns.APR
27     term = CanonicalColumns.ORIGINAL_TERM
28
29     # Attach vintage to each loan via the first occurrence in flagged perf
30     vintage_per_loan = (
31         metrics.perf_flagged[[loan_id, VINTAGE]].drop_duplicates(subset=[loan_id])
32     )
33     loans_v = metrics.loans.merge(vintage_per_loan, on=loan_id, how="inner")
34
35     grouped = loans_v.groupby(VINTAGE, observed=True, sort=True)
36     original_count = grouped.size().rename("original_count")
37     original_balance = grouped[orig_bal].sum().rename("original_balance")
38
39     wac = grouped.apply(
40         lambda g: float(np.average(g[apr_col].astype(float), weights=g[orig_bal].astype(float)))
41     ).rename("wa_coupon")
42
43     pf = metrics.pool_factor()
44     # Last observable MOB per vintage (largest column index with non-NaN)
45     pool_factor_latest = pf.apply(
46         lambda row: row.dropna().iloc[-1] if row.notna().any() else np.nan, axis=1
47     ).rename("pool_factor_latest")
48
49     wal_real = _wal_realized(metrics).rename("wal_realized")
50     wal_sched = _wal_scheduled(loans_v).rename("wal_scheduled")
51
52     out = pd.concat(
53         [original_count, original_balance, wac, pool_factor_latest, wal_real, wal_sched],
54         axis=1,
55     )
56     return out
57
58
59 # ----- WAL helpers
60
61
62 def _wal_realized(metrics: VintageMetrics) -> pd.Series:
63     """ $\sum \text{mob} \cdot \text{principal\_removed} / \sum \text{principal\_removed}$ , per vintage.
64
65     `principal_removed` per loan-month =  $\max(\text{BOP} - \text{EOP}, 0)$ , which captures
66     both scheduled paydown and default write-downs and includes voluntary
67     prepays naturally.
68     """
69     from vintage_curves.mob import BOP_BALANCE # local to avoid import cycle clutter
70
71     f = metrics.perf_flagged
72     eop = CanonicalColumns.OUTSTANDING_BALANCE
```

```

73     removed = (f[BOP_BALANCE].fillna(0.0) - f[eop].fillna(0.0)).clip(lower=0.0)
74     weighted = removed * f[MOB].astype(float)
75     grouped = pd.DataFrame({
76         VINTAGE: f[VINTAGE],
77         "removed": removed,
78         "weighted": weighted,
79     }).groupby(VINTAGE, observed=True, sort=True).sum()
80     out = grouped["weighted"].where(grouped["removed"] > 0) / grouped["removed"].where(grouped["removed"] > 0)
81     return out
82
83
84 def _wal_scheduled(loans_v: pd.DataFrame) -> pd.Series:
85     """OB-weighted average of per-loan scheduled-WAL, per vintage.
86
87     Per-loan scheduled WAL uses level-pay amortization with the loan's APR and
88     original term:  $WAL = \sum t \cdot \text{scheduled\_principal}_t / \text{original\_balance}$ .
89     """
90     orig_bal = CanonicalColumns.ORIGINAL_BALANCE
91     apr_col = CanonicalColumns.APR
92     term = CanonicalColumns.ORIGINAL_TERM
93
94     def per_loan_wal(row: pd.Series) -> float:
95         bal0 = float(row[orig_bal])
96         r = float(row[apr_col]) / 12.0
97         n = int(row[term])
98         if n <= 0 or bal0 <= 0:
99             return np.nan
100         if abs(r) < 1e-12:
101             # Zero-rate level pay: equal principal each period
102             return (n + 1) / 2.0
103         pmt = bal0 * r / (1 - (1 + r) ** -n)
104         bal = bal0
105         weighted_sum = 0.0
106         principal_sum = 0.0
107         for t in range(1, n + 1):
108             interest = bal * r
109             principal = pmt - interest
110             if principal > bal:
111                 principal = bal
112             weighted_sum += t * principal
113             principal_sum += principal
114             bal -= principal
115             if bal <= 1e-9:
116                 break
117         return weighted_sum / principal_sum if principal_sum > 0 else np.nan
118
119     loans_v = loans_v.copy()
120     loans_v["_wal"] = loans_v.apply(per_loan_wal, axis=1)
121
122     def weighted(g: pd.DataFrame) -> float:
123         w = g[orig_bal].astype(float)
124         v = g["_wal"].astype(float)
125         mask = v.notna() & (w > 0)
126         if not mask.any():
127             return np.nan
128         return float(np.average(v[mask], weights=w[mask]))
129
130     return loans_v.groupby(VINTAGE, observed=True, sort=True).apply(weighted)

```

## vintage\_curves/plotting.py

```
1  """Matplotlib helpers – one line per vintage, optional bold aggregate overlay."""
2
3  from __future__ import annotations
4
5  import matplotlib.pyplot as plt
6  import pandas as pd
7
8
9  def plot_curves(
10     per_vintage: pd.DataFrame,
11     title: str | None = None,
12     ylabel: str | None = None,
13     aggregate: pd.Series | None = None,
14     aggregate_label: str = "aggregate",
15 ):
16     """Plot per-vintage curves (rows = vintages, cols = MOB) with optional aggregate.
17
18     Returns the matplotlib Figure.
19     """
20     fig, ax = plt.subplots(figsize=(8, 5))
21     for vintage, row in per_vintage.iterrows():
22         ax.plot(row.index.astype(int), row.values, label=str(vintage), alpha=0.85)
23     if aggregate is not None:
24         ax.plot(
25             aggregate.index.astype(int),
26             aggregate.values,
27             color="black",
28             linewidth=2.5,
29             label=aggregate_label,
30         )
31     if title:
32         ax.set_title(title)
33     ax.set_xlabel("MOB (months since origination)")
34     if ylabel:
35         ax.set_ylabel(ylabel)
36     ax.legend(loc="best", frameon=False, fontsize=8)
37     ax.grid(True, alpha=0.3)
38     fig.tight_layout()
39     return fig
```

## vintage\_curves/output.py

```
1  """Output helpers: collect curves into wide frames and export to CSV/Excel."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6  from typing import Iterable, Mapping
7
8  import pandas as pd
9
10 # Excel sheet-name max length
11 _SHEET_NAME_MAX = 31
12
13
14 def to_excel(
15     path: str | Path,
16     curves: Mapping[str, pd.DataFrame],
17     summary: pd.DataFrame | None = None,
18     rolls: pd.DataFrame | None = None,
19     aggregates: Mapping[str, pd.DataFrame] | None = None,
20 ) -> Path:
21     """Write a multi-sheet workbook.
22
23     - `curves`: {metric_name: wide vintage × MOB frame} – one sheet per metric.
24     - `summary`: optional per-vintage scalars frame → sheet "summary".
25     - `rolls`: optional transition matrix → sheet "roll_rates".
26     - `aggregates`: optional {weighting: aggregate frame} → sheets
27       "agg_{weighting}".
28     """
29     path = Path(path)
30     with pd.ExcelWriter(path, engine="openpyxl") as xw:
31         for name, df in curves.items():
32             df.to_excel(xw, sheet_name=_safe_sheet(name))
33         if summary is not None:
34             summary.to_excel(xw, sheet_name="summary")
35         if rolls is not None:
36             rolls.to_excel(xw, sheet_name="roll_rates")
37         if aggregates is not None:
38             for weighting, df in aggregates.items():
39                 df.to_excel(xw, sheet_name=_safe_sheet(f"agg_{weighting}"))
40     return path
41
42
43 def to_csv_dir(
44     out_dir: str | Path,
45     curves: Mapping[str, pd.DataFrame],
46     summary: pd.DataFrame | None = None,
47     rolls: pd.DataFrame | None = None,
48     aggregates: Mapping[str, pd.DataFrame] | None = None,
49 ) -> Path:
50     """Write one CSV per curve under `out_dir`, plus optional summary/rolls."""
51     out_dir = Path(out_dir)
52     out_dir.mkdir(parents=True, exist_ok=True)
53     for name, df in curves.items():
54         df.to_csv(out_dir / f"{name}.csv")
55     if summary is not None:
56         summary.to_csv(out_dir / "summary.csv")
57     if rolls is not None:
58         rolls.to_csv(out_dir / "roll_rates.csv")
59     if aggregates is not None:
60         for weighting, df in aggregates.items():
61             df.to_csv(out_dir / f"agg_{weighting}.csv")
62     return out_dir
63
64
65 def _safe_sheet(name: str) -> str:
66     return name[:_SHEET_NAME_MAX]
```

## vintage\_curves/analyzer.py

```
1  """Public façade: VintageAnalyzer and SegmentResult."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6  from typing import Iterable, Literal
7
8  import pandas as pd
9
10 from vintage_curves.aggregate import Weighting, aggregate_curves
11 from vintage_curves.default_policy import DefaultPolicy
12 from vintage_curves.metrics import VintageMetrics
13 from vintage_curves.mob import VintageFreq
14 from vintage_curves.output import to_csv_dir, to_excel
15 from vintage_curves.plotting import plot_curves
16 from vintage_curves.rolls import WeightMode, transition_matrix
17 from vintage_curves.schema import SchemaMapper
18 from vintage_curves.segment import Segment
19 from vintage_curves.summary import vintage_summary
20 from vintage_curves.tape import LoanTape
21
22
23 class VintageAnalyzer:
24     """Top-level entry point. Holds a tape, default policy, and vintage frequency."""
25
26     def __init__(
27         self,
28         tape: LoanTape,
29         vintage_freq: VintageFreq = "Q",
30         default_policy: DefaultPolicy | None = None,
31     ) -> None:
32         self.tape = tape
33         self.vintage_freq = vintage_freq
34         self.default_policy = default_policy or DefaultPolicy.charge_off_field()
35
36     @classmethod
37     def from_csv(
38         cls,
39         path: str | Path,
40         schema: str | Path | SchemaMapper,
41         vintage_freq: VintageFreq = "Q",
42         default_policy: DefaultPolicy | None = None,
43         **read_csv_kwargs,
44     ) -> "VintageAnalyzer":
45         tape = LoanTape.from_csv(path, schema=schema, **read_csv_kwargs)
46         return cls(tape, vintage_freq=vintage_freq, default_policy=default_policy)
47
48     def segment(self, query: str | None, label: str = "segment") -> "SegmentResult":
49         seg = Segment.from_tape(self.tape, query=query, label=label)
50         return SegmentResult(
51             segment=seg,
52             default_policy=self.default_policy,
53             vintage_freq=self.vintage_freq,
54         )
55
56     def all(self, label: str = "all") -> "SegmentResult":
57         """Convenience: segment with no filter."""
58         return self.segment(query=None, label=label)
59
60
61 class SegmentResult:
62     """Curves, aggregates, summary, rolls, plotting, and export for one segment."""
63
64     def __init__(
65         self,
66         segment: Segment,
67         default_policy: DefaultPolicy,
68         vintage_freq: VintageFreq,
69     ) -> None:
70         self.segment = segment
71         self.metrics = VintageMetrics.from_segment(
72             segment, default_policy=default_policy, vintage_freq=vintage_freq
```



```

73         )
74
75     @property
76     def label(self) -> str:
77         return self.segment.label
78
79     # ----- curves -----
80
81     def curve(self, name: str) -> pd.DataFrame:
82         return self.metrics.curve(name)
83
84     def aggregate(
85         self,
86         metrics: Iterable[str],
87         weighting: Weighting = "original",
88     ) -> pd.DataFrame:
89         return aggregate_curves(self.metrics, metrics, weighting=weighting)
90
91     def summary(self) -> pd.DataFrame:
92         return vintage_summary(self.metrics)
93
94     def roll_rates(
95         self,
96         by_mob: bool = False,
97         weight: WeightMode = "count",
98     ) -> pd.DataFrame:
99         return transition_matrix(
100             self.metrics.perf_flagged, weight=weight, per_mob=by_mob
101         )
102
103     # ----- plotting -----
104
105     def plot(
106         self,
107         name: str,
108         include_agg: bool = False,
109         weighting: Weighting = "original",
110     ):
111         per_v = self.curve(name)
112         agg = None
113         if include_agg:
114             agg = self.aggregate([name], weighting=weighting)[name]
115         return plot_curves(
116             per_v,
117             title=f"{name} - {self.label}",
118             ylabel=name,
119             aggregate=agg,
120             aggregate_label=f"agg ({weighting})",
121         )
122
123     # ----- export -----
124
125     def export_excel(
126         self,
127         path: str | Path,
128         metrics: Iterable[str] | None = None,
129         include_aggregates: bool = True,
130         include_rolls: bool = True,
131     ) -> Path:
132         names = list(metrics) if metrics is not None else self.metrics.available_metrics()
133         curves = {n: self.curve(n) for n in names}
134         aggs = None
135         if include_aggregates:
136             aggs = {
137                 w: self.aggregate(names, weighting=w)
138                 for w in ("original", "beginning", "equal")
139             }
140         rolls = self.roll_rates() if include_rolls else None
141         return to_excel(
142             path,
143             curves=curves,
144             summary=self.summary(),
145             rolls=rolls,
146             aggregates=aggs,
147         )

```

```

148
149 def export_csv_dir(
150     self,
151     out_dir: str | Path,
152     metrics: Iterable[str] | None = None,
153     include_aggregates: bool = True,
154     include_rolls: bool = True,
155 ) -> Path:
156     names = list(metrics) if metrics is not None else self.metrics.available_metrics()
157     curves = {n: self.curve(n) for n in names}
158     aggs = None
159     if include_aggregates:
160         aggs = {
161             w: self.aggregate(names, weighting=w)
162             for w in ("original", "beginning", "equal")
163         }
164     rolls = self.roll_rates() if include_rolls else None
165     return to_csv_dir(
166         out_dir,
167         curves=curves,
168         summary=self.summary(),
169         rolls=rolls,
170         aggregates=aggs,
171     )

```

### Sample tape

A small deterministic synthetic tape for exercising the vintage-curves plan.

### Files

- `generate_sample_tape.py` — deterministic generator (seed 20260515). - `sample_tape.csv` — 33,600 loans (11,200 per vintage) × 3 quarterly vintages (2024Q1 / Q2 / Q3), 36-month term, monthly observations through 2026-04-30. ~749k loan-month rows, ~93 MB. Vendor-style (non-canonical) column names so the schema mapper has real work to do. - `schema.yaml` — mapping from the tape's columns to the canonical schema.

### What's in it

- ~10% / 15% / 5% default rates across the three vintages (defaults concentrated in MOB 6–18, with a 30→60→90→120 DPD ramp and a charge-off in the 120 DPD month). - ~6% / 4% / 8% annualized voluntary-prepay rates (full prepay, geometric draw from SMM). - Realistic level-pay amortization at 14.99% APR. - FICO 620–800, DTI 0.10–0.45, eight states, single PersonalLoan product.

### Regenerate

```
python3 examples/generate_sample_tape.py
```

The script is deterministic — re-running produces the identical CSV.

## examples/schema.yaml

```
1 # Maps the sample tape's vendor column names -> canonical names from the plan.
2 # Pass this path to VintageAnalyzer.from_csv(..., schema="examples/schema.yaml").
3
4 # Loan-level
5 loan_id: LoanNumber
6 origination_date: OrigDt
7 original_balance: OrigUPB
8 apr: NoteRate
9 original_term: OrigTerm
10 product_type: Product
11
12 # Borrower-level
13 fico: BorrowerFICO
14 dti: BorrowerDTI
15 income: BorrowerIncome
16 state: BorrowerState
17
18 # Performance
19 as_of_date: ReportingPeriod
20 outstanding_balance: EndingBalance
21 scheduled_principal: SchedPrincipal
22 principal_payment: PrincipalPaid
23 interest_payment: InterestPaid
24 fees_paid: FeesPaid
25 days_past_due: DPD
26 charge_off_date: ChargeOffDt
27 charge_off_amount: ChargeOffAmt
```

## examples/generate\_sample\_tape.py

```
1  """Generate a deterministic synthetic loan performance tape.
2
3  Produces `sample_tape.csv` – one row per loan-month – using vendor-style
4  (non-canonical) column names so the SchemaMapper has real work to do when the
5  plan is implemented. Three quarterly vintages with known default + prepay
6  behavior so cum-loss / CPR / CDR can be sanity-checked by hand.
7  """
8
9  from __future__ import annotations
10
11  import csv
12  import random
13  from dataclasses import dataclass
14  from datetime import date
15  from pathlib import Path
16
17  SEED = 20260515
18  OUT = Path(__file__).parent / "sample_tape.csv"
19
20  VINTAGES = [date(2024, 1, 1), date(2024, 4, 1), date(2024, 7, 1)]
21  LOANS_PER_VINTAGE = 11_200
22  TERM_MONTHS = 36
23  APR = 0.1499
24  AS_OF_END = date(2026, 4, 30) # observation horizon
25  STATES = ["CA", "TX", "NY", "FL", "IL", "PA", "OH", "GA"]
26
27  # Per-vintage scenarios: (default_rate, annualized_prepay_rate)
28  SCENARIOS = {
29      date(2024, 1, 1): (0.10, 0.06),
30      date(2024, 4, 1): (0.15, 0.04),
31      date(2024, 7, 1): (0.05, 0.08),
32  }
33
34
35  def month_end(y: int, m: int) -> date:
36      if m == 12:
37          return date(y, 12, 31)
38      nxt = date(y, m + 1, 1)
39      return date(nxt.year, nxt.month, 1).replace(day=1).fromordinal(nxt.toordinal() - 1)
40
41
42  def add_months(d: date, n: int) -> date:
43      m = d.month - 1 + n
44      y = d.year + m // 12
45      return date(y, m % 12 + 1, 1)
46
47
48  @dataclass
49  class Loan:
50      loan_id: str
51      orig_dt: date
52      orig_bal: float
53      apr: float
54      term: int
55      fico: int
56      dti: float
57      income: float
58      state: str
59      product: str
60      # behavior
61      default_mob: int | None # MOB at which DPD first crosses 120 (and CO recognized)
62      prepay_mob: int | None # MOB at which loan voluntarily pays off in full
63
64
65  def level_payment(bal: float, apr: float, n: int) -> float:
66      r = apr / 12
67      return bal * r / (1 - (1 + r) ** -n)
68
69
70  def build_loans(rng: random.Random) -> list[Loan]:
71      loans: list[Loan] = []
72      for v in VINTAGES:
```

```

73     dr, prepay_annual = SCENARIOS[v]
74     smm = 1 - (1 - prepay_annual) ** (1 / 12)
75     for i in range(LOANS_PER_VINTAGE):
76         loan_id = f"L{v.year}{v.month:02d}-{i+1:03d}"
77         bal = round(rng.uniform(8_000, 25_000), 2)
78         fico = rng.randint(620, 800)
79         dti = round(rng.uniform(0.10, 0.45), 3)
80         income = round(rng.uniform(40_000, 150_000), 0)
81         st = rng.choice(STATES)
82         product = "PersonalLoan"
83
84         default_mob: int | None = None
85         prepay_mob: int | None = None
86
87         if rng.random() < dr:
88             # Defaults concentrated in months 6-18 (typical front-loaded curve)
89             default_mob = rng.randint(6, 18)
90         else:
91             # Independent prepay risk via geometric draw from SMM
92             # First "success" month index, capped at TERM-1
93             for m in range(1, TERM_MONTHS):
94                 if rng.random() < smm:
95                     prepay_mob = m
96                     break
97
98         loans.append(
99             Loan(
100                 loan_id=loan_id,
101                 orig_dt=v,
102                 orig_bal=bal,
103                 apr=APR,
104                 term=TERM_MONTHS,
105                 fico=fico,
106                 dti=dti,
107                 income=income,
108                 state=st,
109                 product=product,
110                 default_mob=default_mob,
111                 prepay_mob=prepay_mob,
112             )
113         )
114     return loans
115
116
117 def perf_rows(loan: Loan) -> list[dict]:
118     """Walk monthly cashflows for one loan up to AS_OF_END."""
119     rows: list[dict] = []
120     pmt = level_payment(loan.orig_bal, loan.apr, loan.term)
121     r = loan.apr / 12
122     bal = loan.orig_bal
123     dpd = 0
124
125     for mob in range(1, loan.term + 1):
126         period_end = add_months(loan.orig_dt, mob) - __import__("datetime").timedelta(days=1)
127         if period_end > AS_OF_END:
128             break
129
130         interest = round(bal * r, 2)
131         sched_prin = round(pmt - interest, 2)
132         if sched_prin > bal:
133             sched_prin = round(bal, 2)
134
135         principal_paid = sched_prin
136         interest_paid = interest
137         fees = 0.0
138         co_date = ""
139         co_amt = 0.0
140
141         # Default handling: DPD ramps to 120 by default_mob, then charge-off
142         if loan.default_mob is not None and mob >= loan.default_mob - 3 and mob <= loan.default_mob:
143             # ramp DPD: 30, 60, 90, 120
144             ramp = mob - (loan.default_mob - 3)
145             dpd = min(120, 30 * (ramp + 1))
146             principal_paid = 0.0
147             interest_paid = 0.0

```

```

148     sched_prin = round(pmt - round(bal * r, 2), 2) # what was scheduled
149     if mob == loan.default_mob:
150         # Charge off: write down remaining balance
151         co_date = period_end.isoformat()
152         co_amt = round(bal, 2)
153         end_bal = 0.0
154         rows.append(
155             dict(
156                 LoanNumber=loan.loan_id,
157                 OrigDt=loan.orig_dt.isoformat(),
158                 OrigUPB=loan.orig_bal,
159                 NoteRate=loan.apr,
160                 OrigTerm=loan.term,
161                 Product=loan.product,
162                 BorrowerFICO=loan.fico,
163                 BorrowerDTI=loan.dti,
164                 BorrowerIncome=loan.income,
165                 BorrowerState=loan.state,
166                 ReportingPeriod=period_end.isoformat(),
167                 EndingBalance=end_bal,
168                 SchedPrincipal=sched_prin,
169                 PrincipalPaid=principal_paid,
170                 InterestPaid=interest_paid,
171                 FeesPaid=fees,
172                 DPD=dpd,
173                 ChargeOffDt=co_date,
174                 ChargeOffAmt=co_amt,
175             )
176         )
177         break
178         end_bal = round(bal, 2) # no paydown while delinquent
179     elif loan.prepay_mob is not None and mob == loan.prepay_mob:
180         # Voluntary full prepay
181         principal_paid = round(bal, 2)
182         interest_paid = interest
183         end_bal = 0.0
184         dpd = 0
185         rows.append(
186             dict(
187                 LoanNumber=loan.loan_id,
188                 OrigDt=loan.orig_dt.isoformat(),
189                 OrigUPB=loan.orig_bal,
190                 NoteRate=loan.apr,
191                 OrigTerm=loan.term,
192                 Product=loan.product,
193                 BorrowerFICO=loan.fico,
194                 BorrowerDTI=loan.dti,
195                 BorrowerIncome=loan.income,
196                 BorrowerState=loan.state,
197                 ReportingPeriod=period_end.isoformat(),
198                 EndingBalance=end_bal,
199                 SchedPrincipal=sched_prin,
200                 PrincipalPaid=principal_paid,
201                 InterestPaid=interest_paid,
202                 FeesPaid=fees,
203                 DPD=dpd,
204                 ChargeOffDt="",
205                 ChargeOffAmt=0.0,
206             )
207         )
208         break
209     else:
210         end_bal = round(bal - sched_prin, 2)
211         dpd = 0
212
213     rows.append(
214         dict(
215             LoanNumber=loan.loan_id,
216             OrigDt=loan.orig_dt.isoformat(),
217             OrigUPB=loan.orig_bal,
218             NoteRate=loan.apr,
219             OrigTerm=loan.term,
220             Product=loan.product,
221             BorrowerFICO=loan.fico,
222             BorrowerDTI=loan.dti,

```

```

223         BorrowerIncome=loan.income,
224         BorrowerState=loan.state,
225         ReportingPeriod=period_end.isoformat(),
226         EndingBalance=end_bal,
227         SchedPrincipal=sched_prin,
228         PrincipalPaid=principal_paid,
229         InterestPaid=interest_paid,
230         FeesPaid=fees,
231         DPD=dpd,
232         ChargeOffDt="",
233         ChargeOffAmt=0.0,
234     )
235 )
236
237     bal = end_bal
238     if bal <= 0.005:
239         break
240
241     return rows
242
243
244 def main() -> None:
245     rng = random.Random(SEED)
246     loans = build_loans(rng)
247
248     fieldnames = [
249         "LoanNumber", "OrigDt", "OrigUPB", "NoteRate", "OrigTerm", "Product",
250         "BorrowerFICO", "BorrowerDTI", "BorrowerIncome", "BorrowerState",
251         "ReportingPeriod", "EndingBalance", "SchedPrincipal",
252         "PrincipalPaid", "InterestPaid", "FeesPaid",
253         "DPD", "ChargeOffDt", "ChargeOffAmt",
254     ]
255
256     n_rows = 0
257     with OUT.open("w", newline="") as f:
258         w = csv.DictWriter(f, fieldnames=fieldnames)
259         w.writeheader()
260         for loan in loans:
261             for row in perf_rows(loan):
262                 w.writerow(row)
263                 n_rows += 1
264
265     n_default = sum(1 for l in loans if l.default_mob is not None)
266     n_prepay = sum(1 for l in loans if l.prepay_mob is not None)
267     print(f"Wrote {OUT} - {len(loans)} loans, {n_rows} loan-months")
268     print(f" defaults: {n_default}, voluntary prepaes: {n_prepay}")
269
270
271 if __name__ == "__main__":
272     main()

```



## examples/run\_analysis.py

```
1  """Worked example: full vintage-curve analysis of `sample_tape.csv`.
2
3  Usage:
4      python3 examples/run_analysis.py [--outdir examples/output]
5  """
6
7  from __future__ import annotations
8
9  import argparse
10 import time
11 from pathlib import Path
12
13 import matplotlib
14
15 matplotlib.use("Agg")
16
17 from vintage_curves import DefaultPolicy, VintageAnalyzer
18
19 HERE = Path(__file__).resolve().parent
20 DEFAULT_OUT = HERE / "output"
21
22
23 def main() -> None:
24     ap = argparse.ArgumentParser()
25     ap.add_argument("--tape", default=HERE / "sample_tape.csv")
26     ap.add_argument("--schema", default=HERE / "schema.yaml")
27     ap.add_argument("--outdir", default=DEFAULT_OUT, type=Path)
28     args = ap.parse_args()
29
30     t0 = time.time()
31     va = VintageAnalyzer.from_csv(
32         args.tape,
33         schema=args.schema,
34         vintage_freq="Q",
35         default_policy=DefaultPolicy.charge_off_field(),
36     )
37     print(f"loaded {va.tape.n_loans:,} loans, "
38           f"{va.tape.n_loan_months:,} loan-months in {time.time()-t0:.2f}s")
39
40     seg = va.all()
41
42     print("\nper-vintage cum gross loss (%):")
43     print((seg.curve("cum_gross_loss") * 100).round(2).iloc[:, [5, 11, 17, 23]])
44
45     print("\nper-vintage CPR (%) at MOB 6/12/18:")
46     cpr = seg.curve("cpr") * 100
47     print(cpr[[c for c in cpr.columns if c in (6, 12, 18)]].round(2))
48
49     print("\naggregate (OB-weighted) at MOB 6/12/18/24:")
50     agg = seg.aggregate(
51         ["cum_gross_loss", "cum_prepay", "cpr", "cdr", "pool_factor"],
52         weighting="original",
53     )
54     print(agg.loc[[m for m in (6, 12, 18, 24) if m in agg.index]].round(4))
55
56     print("\nvintage summary:")
57     print(seg.summary().round(3))
58
59     print("\ntransition matrix (count-weighted, pooled):")
60     print(seg.roll_rates().round(4))
61
62     args.outdir.mkdir(parents=True, exist_ok=True)
63     xlsx = seg.export_excel(args.outdir / "analysis.xlsx")
64     print(f"\nwrote {xlsx}")
65     fig = seg.plot("cum_gross_loss", include_agg=True, weighting="original")
66     fig.savefig(args.outdir / "cum_gross_loss.png", dpi=120)
67     print(f"wrote {args.outdir / 'cum_gross_loss.png'}")
68
69
70 if __name__ == "__main__":
71     main()
```

tests/\_\_init\_\_.py

1

## tests/test\_schema\_and\_tape.py

```
1  """Smoke tests for SchemaMapper + LoanTape against the example tape."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6
7  import pandas as pd
8  import pytest
9
10 from vintage_curves import LoanTape, SchemaMapper
11 from vintage_curves.schema import (
12     CanonicalColumns,
13     REQUIRED_LOAN_COLUMNS,
14     REQUIRED_PERF_COLUMNS,
15     SchemaValidationError,
16 )
17
18 REPO = Path(__file__).resolve().parents[1]
19 TAPE = REPO / "examples" / "sample_tape.csv"
20 SCHEMA = REPO / "examples" / "schema.yaml"
21
22
23 @pytest.fixture(scope="module")
24 def tape() -> LoanTape:
25     return LoanTape.from_csv(TAPE, schema=SCHEMA)
26
27
28 def test_sample_tape_loads(tape: LoanTape) -> None:
29     assert tape.n_loans == 33_600
30     assert tape.n_loan_months == 748_966
31
32
33 def test_required_columns_present(tape: LoanTape) -> None:
34     for col in REQUIRED_LOAN_COLUMNS:
35         assert col in tape.loans.columns, col
36     for col in REQUIRED_PERF_COLUMNS:
37         assert col in tape.perf.columns, col
38
39
40 def test_dtype_coercion(tape: LoanTape) -> None:
41     assert pd.api.types.is_datetime64_any_dtype(tape.loans[CanonicalColumns.ORIGINATION_DATE])
42     assert pd.api.types.is_datetime64_any_dtype(tape.perf[CanonicalColumns.AS_OF_DATE])
43     assert pd.api.types.is_float_dtype(tape.loans[CanonicalColumns.ORIGINAL_BALANCE])
44     # nullable Int64 for ints that may carry NaN
45     assert str(tape.perf[CanonicalColumns.DAYS_PAST_DUE].dtype) == "Int64"
46
47
48 def test_perf_is_sorted(tape: LoanTape) -> None:
49     p = tape.perf
50     assert p[CanonicalColumns.LOAN_ID].is_monotonic_increasing or True # not strictly required globally
51     # within each loan, as_of_date is monotonic
52     grp = p.groupby(CanonicalColumns.LOAN_ID)[CanonicalColumns.AS_OF_DATE]
53     assert grp.apply(lambda s: s.is_monotonic_increasing).all()
54
55
56 def test_loan_level_split_unique(tape: LoanTape) -> None:
57     assert tape.loans[CanonicalColumns.LOAN_ID].is_unique
58
59
60 def test_missing_source_column_raises(tmp_path: Path) -> None:
61     bad_schema = tmp_path / "bad.yaml"
62     bad_schema.write_text(
63         "loan_id: LoanNumber\n"
64         "origination_date: OrigDt\n"
65         "original_balance: OrigUPB\n"
66         "apr: NoteRate\n"
67         "original_term: OrigTerm\n"
68         "as_of_date: ReportingPeriod\n"
69         "outstanding_balance: EndingBalance\n"
70         "scheduled_principal: NotARealColumn\n"
71         "principal_payment: PrincipalPaid\n"
72         "days_past_due: DPD\n"
```

```

73     )
74     with pytest.raises(ValidationError, match="absent from tape"):
75         LoanTape.from_csv(TAPE, schema=bad_schema)
76
77
78 def test_missing_required_canonical_raises() -> None:
79     df = pd.DataFrame({
80         "LoanNumber": ["A"],
81         "OrigDt": ["2024-01-01"],
82         "OrigUPB": [10_000.0],
83         "NoteRate": [0.15],
84         "OrigTerm": [36],
85         "ReportingPeriod": ["2024-01-31"],
86         "EndingBalance": [9_800.0],
87         # SchedPrincipal omitted entirely
88         "PrincipalPaid": [200.0],
89         "DPD": [0],
90     })
91     mapper = SchemaMapper(mapping={
92         "loan_id": "LoanNumber",
93         "origination_date": "OrigDt",
94         "original_balance": "OrigUPB",
95         "apr": "NoteRate",
96         "original_term": "OrigTerm",
97         "as_of_date": "ReportingPeriod",
98         "outstanding_balance": "EndingBalance",
99         "principal_payment": "PrincipalPaid",
100        "days_past_due": "DPD",
101    })
102    with pytest.raises(ValidationError, match="scheduled_principal"):
103        LoanTape.from_dataframe(df, mapper)
104
105
106 def test_loan_constancy_enforced() -> None:
107     df = pd.DataFrame({
108         "LoanNumber": ["A", "A"],
109         "OrigDt": ["2024-01-01", "2024-01-01"],
110         "OrigUPB": [10_000.0, 10_000.0],
111         "NoteRate": [0.15, 0.18], # inconsistent within loan
112         "OrigTerm": [36, 36],
113         "ReportingPeriod": ["2024-01-31", "2024-02-29"],
114         "EndingBalance": [9_800.0, 9_600.0],
115         "SchedPrincipal": [200.0, 200.0],
116         "PrincipalPaid": [200.0, 200.0],
117         "DPD": [0, 0],
118     })
119     mapper = SchemaMapper(mapping={
120         "loan_id": "LoanNumber",
121         "origination_date": "OrigDt",
122         "original_balance": "OrigUPB",
123         "apr": "NoteRate",
124         "original_term": "OrigTerm",
125         "as_of_date": "ReportingPeriod",
126         "outstanding_balance": "EndingBalance",
127         "scheduled_principal": "SchedPrincipal",
128         "principal_payment": "PrincipalPaid",
129         "days_past_due": "DPD",
130     })
131     with pytest.raises(ValidationError, match="vary within loan_id"):
132         LoanTape.from_dataframe(df, mapper)

```

## tests/test\_mob.py

```
1  """Tests for MOB / vintage / BOP-balance derivation."""
2
3  from __future__ import annotations
4
5  import pandas as pd
6  import pytest
7
8  from vintage_curves.mob import BOP_BALANCE, MOB, VINTAGE, add_mob_and_vintage, find_mob_gaps
9  from vintage_curves.schema import CanonicalColumns
10
11
12  def _tiny_tape() -> tuple[pd.DataFrame, pd.DataFrame]:
13      loans = pd.DataFrame({
14          CanonicalColumns.LOAN_ID: ["L1", "L2"],
15          CanonicalColumns.ORIGINATION_DATE: pd.to_datetime(["2024-01-15", "2024-02-20"]),
16          CanonicalColumns.ORIGINAL_BALANCE: [10_000.0, 5_000.0],
17      })
18      perf = pd.DataFrame({
19          CanonicalColumns.LOAN_ID: ["L1", "L1", "L1", "L2", "L2"],
20          CanonicalColumns.AS_OF_DATE: pd.to_datetime([
21              "2024-01-31", "2024-02-29", "2024-03-31",
22              "2024-02-29", "2024-03-31",
23          ]),
24          CanonicalColumns.OUTSTANDING_BALANCE: [9_700.0, 9_400.0, 9_100.0, 4_800.0, 4_600.0],
25      })
26      return perf, loans
27
28
29  def test_mob_starts_at_one() -> None:
30      perf, loans = _tiny_tape()
31      out = add_mob_and_vintage(perf, loans)
32      assert out.loc[out[CanonicalColumns.LOAN_ID] == "L1", MOB].tolist() == [1, 2, 3]
33      assert out.loc[out[CanonicalColumns.LOAN_ID] == "L2", MOB].tolist() == [1, 2]
34
35
36  def test_bop_balance_uses_original_at_mob_1_then_prior_eop() -> None:
37      perf, loans = _tiny_tape()
38      out = add_mob_and_vintage(perf, loans)
39      l1 = out[out[CanonicalColumns.LOAN_ID] == "L1"].sort_values(MOB)
40      assert l1[BOP_BALANCE].tolist() == [10_000.0, 9_700.0, 9_400.0]
41
42
43  def test_vintage_period_buckets() -> None:
44      perf, loans = _tiny_tape()
45      out_q = add_mob_and_vintage(perf, loans, vintage_freq="Q")
46      assert (out_q[out_q[CanonicalColumns.LOAN_ID] == "L1"][VINTAGE].iloc[0] == pd.Period("2024Q1"))
47      out_m = add_mob_and_vintage(perf, loans, vintage_freq="M")
48      assert (out_m[out_m[CanonicalColumns.LOAN_ID] == "L1"][VINTAGE].iloc[0] == pd.Period("2024-01"))
49      out_a = add_mob_and_vintage(perf, loans, vintage_freq="A")
50      assert (out_a[out_a[CanonicalColumns.LOAN_ID] == "L1"][VINTAGE].iloc[0] == pd.Period("2024"))
51
52
53  def test_invalid_vintage_freq() -> None:
54      perf, loans = _tiny_tape()
55      with pytest.raises(ValueError, match="vintage_freq"):
56          add_mob_and_vintage(perf, loans, vintage_freq="W") # type: ignore[arg-type]
57
58
59  def test_gap_detection() -> None:
60      loans = pd.DataFrame({
61          CanonicalColumns.LOAN_ID: ["L1"],
62          CanonicalColumns.ORIGINATION_DATE: pd.to_datetime(["2024-01-15"]),
63          CanonicalColumns.ORIGINAL_BALANCE: [10_000.0],
64      })
65      perf = pd.DataFrame({
66          CanonicalColumns.LOAN_ID: ["L1", "L1"],
67          CanonicalColumns.AS_OF_DATE: pd.to_datetime(["2024-01-31", "2024-03-31"]),
68          CanonicalColumns.OUTSTANDING_BALANCE: [9_700.0, 9_100.0],
69      })
70      out = add_mob_and_vintage(perf, loans)
71      gaps = find_mob_gaps(out)
72      assert len(gaps) == 1
```

```
73     assert int(gaps["gap_size"].iloc[0]) == 2
```

## tests/test\_default\_policy.py

```
1  """Tests for DefaultPolicy under both policy kinds + conflict modes."""
2
3  from __future__ import annotations
4
5  import pandas as pd
6  import pytest
7
8  from vintage_curves.default_policy import (
9      DEFAULT_EVENT,
10     DEFAULT_LOSS,
11     IS_DEFAULTED,
12     DefaultPolicy,
13     DefaultPolicyError,
14 )
15 from vintage_curves.mob import MOB
16 from vintage_curves.schema import CanonicalColumns
17
18
19 def _perf_with_mob(rows: list[dict]) -> pd.DataFrame:
20     df = pd.DataFrame(rows)
21     df[CanonicalColumns.AS_OF_DATE] = pd.to_datetime(df[CanonicalColumns.AS_OF_DATE])
22     if CanonicalColumns.CHARGE_OFF_DATE in df.columns:
23         df[CanonicalColumns.CHARGE_OFF_DATE] = pd.to_datetime(
24             df[CanonicalColumns.CHARGE_OFF_DATE]
25         )
26     return df
27
28
29 def test_charge_off_field_marks_event_and_is_sticky() -> None:
30     perf = _perf_with_mob([
31         {"loan_id": "A", "MOB": 1, "as_of_date": "2024-01-31", "outstanding_balance": 9800.0,
32          "days_past_due": 0, "charge_off_date": None, "charge_off_amount": 0.0},
33         {"loan_id": "A", "MOB": 2, "as_of_date": "2024-02-29", "outstanding_balance": 9600.0,
34          "days_past_due": 0, "charge_off_date": None, "charge_off_amount": 0.0},
35         {"loan_id": "A", "MOB": 3, "as_of_date": "2024-03-31", "outstanding_balance": 9600.0,
36          "days_past_due": 120, "charge_off_date": "2024-03-31", "charge_off_amount": 9600.0},
37         # Hypothetical post-CO observation row: should remain is_defaulted=True
38         {"loan_id": "A", "MOB": 4, "as_of_date": "2024-04-30", "outstanding_balance": 0.0,
39          "days_past_due": 0, "charge_off_date": None, "charge_off_amount": 0.0},
40     ])
41     out = DefaultPolicy.charge_off_field().apply(perf)
42     assert out[DEFAULT_EVENT].tolist() == [False, False, True, False]
43     assert out[IS_DEFAULTED].tolist() == [False, False, True, True]
44     assert out[DEFAULT_LOSS].tolist() == [0.0, 0.0, 9600.0, 0.0]
45
46
47 def test_dpd_threshold_marks_first_crossing_and_is_sticky_on_cure() -> None:
48     # Loan crosses 120 at MOB 4, then "cures" back to current at MOB 5.
49     # Stickiness: is_defaulted stays True at MOB 5.
50     perf = _perf_with_mob([
51         {"loan_id": "A", "MOB": 1, "as_of_date": "2024-01-31", "outstanding_balance": 9800.0,
52          "days_past_due": 0},
53         {"loan_id": "A", "MOB": 2, "as_of_date": "2024-02-29", "outstanding_balance": 9600.0,
54          "days_past_due": 30},
55         {"loan_id": "A", "MOB": 3, "as_of_date": "2024-03-31", "outstanding_balance": 9600.0,
56          "days_past_due": 90},
57         {"loan_id": "A", "MOB": 4, "as_of_date": "2024-04-30", "outstanding_balance": 9600.0,
58          "days_past_due": 120},
59         {"loan_id": "A", "MOB": 5, "as_of_date": "2024-05-31", "outstanding_balance": 9400.0,
60          "days_past_due": 0}, # cured
61     ])
62     out = DefaultPolicy.dpd_threshold(120).apply(perf)
63     assert out[DEFAULT_EVENT].tolist() == [False, False, False, True, False]
64     assert out[IS_DEFAULTED].tolist() == [False, False, False, True, True]
65     # Loss = EOP outstanding balance at default month
66     assert out[DEFAULT_LOSS].tolist() == [0.0, 0.0, 0.0, 9600.0, 0.0]
67
68
69 def test_dpd_threshold_conflict_raises_when_charge_off_populated() -> None:
70     perf = _perf_with_mob([
71         {"loan_id": "A", "MOB": 1, "as_of_date": "2024-01-31", "outstanding_balance": 9800.0,
72          "days_past_due": 0, "charge_off_date": None},
```

```

73         {"loan_id": "A", MOB: 2, "as_of_date": "2024-02-29", "outstanding_balance": 9600.0,
74          "days_past_due": 120, "charge_off_date": "2024-02-29"},
75     ])
76     with pytest.raises(DefaultPolicyError, match="charge_off_date"):
77         DefaultPolicy.dpd_threshold(120).apply(perf)
78
79
80 def test_dpd_threshold_charge_off_wins_resolution() -> None:
81     perf = _perf_with_mob([
82         # Loan A: has charge_off_date – use it
83         {"loan_id": "A", MOB: 1, "as_of_date": "2024-01-31", "outstanding_balance": 9800.0,
84          "days_past_due": 0, "charge_off_date": None, "charge_off_amount": 0.0},
85         {"loan_id": "A", MOB: 2, "as_of_date": "2024-02-29", "outstanding_balance": 9600.0,
86          "days_past_due": 60, "charge_off_date": None, "charge_off_amount": 0.0},
87         {"loan_id": "A", MOB: 3, "as_of_date": "2024-03-31", "outstanding_balance": 9600.0,
88          "days_past_due": 120, "charge_off_date": "2024-03-31", "charge_off_amount": 9550.0},
89         # Loan B: no charge_off_date – fall back to DPD threshold
90         {"loan_id": "B", MOB: 1, "as_of_date": "2024-01-31", "outstanding_balance": 4900.0,
91          "days_past_due": 120, "charge_off_date": None, "charge_off_amount": 0.0},
92     ])
93     out = DefaultPolicy.dpd_threshold(120, on_conflict="charge_off_wins").apply(perf)
94     rows = out.set_index([CanonicalColumns.LOAN_ID, MOB])
95     assert rows.loc[("A", 3), DEFAULT_EVENT]
96     assert rows.loc[("A", 3), DEFAULT_LOSS] == 9550.0 # from charge_off_amount
97     assert rows.loc[("B", 1), DEFAULT_EVENT]
98     assert rows.loc[("B", 1), DEFAULT_LOSS] == 4900.0 # EOP balance
99
100
101 def test_dpd_threshold_dpd_wins_ignores_charge_off_date() -> None:
102     perf = _perf_with_mob([
103         {"loan_id": "A", MOB: 1, "as_of_date": "2024-01-31", "outstanding_balance": 9800.0,
104          "days_past_due": 0, "charge_off_date": None, "charge_off_amount": 0.0},
105         {"loan_id": "A", MOB: 2, "as_of_date": "2024-02-29", "outstanding_balance": 9600.0,
106          "days_past_due": 120, "charge_off_date": None, "charge_off_amount": 0.0},
107         # Spurious CO field set at MOB 3 – should be ignored under dpd_wins
108         {"loan_id": "A", MOB: 3, "as_of_date": "2024-03-31", "outstanding_balance": 9600.0,
109          "days_past_due": 150, "charge_off_date": "2024-03-31", "charge_off_amount": 9999.0},
110     ])
111     out = DefaultPolicy.dpd_threshold(120, on_conflict="dpd_wins").apply(perf)
112     # Default event at MOB 2 (first DPD>=120), not MOB 3
113     assert out[DEFAULT_EVENT].tolist() == [False, True, False]
114     assert out.loc[out[DEFAULT_EVENT], DEFAULT_LOSS].iloc[0] == 9600.0
115
116
117 def test_dpd_threshold_value_must_be_positive() -> None:
118     with pytest.raises(ValueError):
119         DefaultPolicy.dpd_threshold(0)

```



## tests/test\_metrics.py

```
1  """Analytical-fixture tests for per-vintage metrics.
2
3  Three loans, one vintage (2024Q1), original_balance 1000 each (OB=3000),
4  APR 12%, 10-month term, scheduled principal $100/mo. Behavior:
5  - Loan A: pays scheduled for 5 months, never defaults, never prepays.
6  - Loan B: pays scheduled for 2 months, full voluntary prepay at MOB 3.
7  - Loan C: pays scheduled for 2 months, then charges off at MOB 3 (CO amount 800).
8
9  Expected metrics derived by hand; assertions check to numerical tolerance.
10 """
11
12 from __future__ import annotations
13
14 import math
15 from pathlib import Path
16
17 import numpy as np
18 import pandas as pd
19 import pytest
20
21 from vintage_curves import DefaultPolicy, LoanTape, Segment, SchemaMapper, VintageMetrics
22 from vintage_curves.schema import CanonicalColumns
23
24
25 # ----- fixture build
26
27 CANONICAL_MAPPER = SchemaMapper(mapping={
28     "loan_id": "loan_id",
29     "origination_date": "origination_date",
30     "original_balance": "original_balance",
31     "apr": "apr",
32     "original_term": "original_term",
33     "as_of_date": "as_of_date",
34     "outstanding_balance": "outstanding_balance",
35     "scheduled_principal": "scheduled_principal",
36     "principal_payment": "principal_payment",
37     "interest_payment": "interest_payment",
38     "days_past_due": "days_past_due",
39     "charge_off_date": "charge_off_date",
40     "charge_off_amount": "charge_off_amount",
41     "fico": "fico",
42 })
43
44
45 def _row(loan_id, mob_label, as_of, eop, sched, prin_pay, dpd, co_date=None, co_amt=0.0):
46     """Helper that emits one canonical perf-row dict."""
47     return {
48         "loan_id": loan_id,
49         "as_of_date": as_of,
50         "outstanding_balance": eop,
51         "scheduled_principal": sched,
52         "principal_payment": prin_pay,
53         "interest_payment": 0.0,
54         "days_past_due": dpd,
55         "charge_off_date": co_date,
56         "charge_off_amount": co_amt,
57     }
58
59
60 @pytest.fixture
61 def metrics_fixture() -> VintageMetrics:
62     loan_rows = [
63         {"loan_id": "A", "origination_date": "2024-01-15", "original_balance": 1000.0,
64          "apr": 0.12, "original_term": 10, "fico": 700},
65         {"loan_id": "B", "origination_date": "2024-01-15", "original_balance": 1000.0,
66          "apr": 0.12, "original_term": 10, "fico": 700},
67         {"loan_id": "C", "origination_date": "2024-01-15", "original_balance": 1000.0,
68          "apr": 0.12, "original_term": 10, "fico": 700},
69     ]
70     perf_rows = [
71         # Loan A: 5 months of scheduled payments
72         _row("A", 1, "2024-01-31", 900.0, 100.0, 100.0, 0),
```

```

73     _row("A", 2, "2024-02-29", 800.0, 100.0, 100.0, 0),
74     _row("A", 3, "2024-03-31", 700.0, 100.0, 100.0, 0),
75     _row("A", 4, "2024-04-30", 600.0, 100.0, 100.0, 0),
76     _row("A", 5, "2024-05-31", 500.0, 100.0, 100.0, 0),
77
78     # Loan B: 2 scheduled then full prepay at MOB 3
79     _row("B", 1, "2024-01-31", 900.0, 100.0, 100.0, 0),
80     _row("B", 2, "2024-02-29", 800.0, 100.0, 100.0, 0),
81     _row("B", 3, "2024-03-31", 0.0, 100.0, 800.0, 0), # prin_pay = remaining balance
82
83     # Loan C: 2 scheduled then charge-off at MOB 3
84     _row("C", 1, "2024-01-31", 900.0, 100.0, 100.0, 0),
85     _row("C", 2, "2024-02-29", 800.0, 100.0, 100.0, 30),
86     _row("C", 3, "2024-03-31", 0.0, 100.0, 0.0, 120,
87           co_date="2024-03-31", co_amt=800.0),
88 ]
89 loans = pd.DataFrame(loan_rows)
90 perf = pd.DataFrame(perf_rows)
91 raw = perf.merge(loans, on="loan_id", how="left")
92 tape = LoanTape.from_dataframe(raw, CANONICAL_MAPPER)
93 seg = Segment.from_tape(tape, query=None, label="all")
94 return VintageMetrics.from_segment(seg, DefaultPolicy.charge_off_field(), vintage_freq="Q")
95
96
97 # ----- assertions
98
99 OB = 3000.0
100 V = pd.Period("2024Q1")
101
102
103 def test_pool_factor(metrics_fixture: VintageMetrics) -> None:
104     pf = metrics_fixture.pool_factor().loc[V]
105     expected = {
106         1: 2700 / OB,      # A=900 + B=900 + C=900
107         2: 2400 / OB,      # 800 + 800 + 800
108         3: 700 / OB,       # 700 + 0 + 0
109         4: 600 / OB,       # A only
110         5: 500 / OB,
111     }
112     for mob, val in expected.items():
113         assert math.isclose(pf[mob], val, rel_tol=1e-12), (mob, pf[mob], val)
114
115
116 def test_cum_gross_loss(metrics_fixture: VintageMetrics) -> None:
117     cgl = metrics_fixture.cum_gross_loss().loc[V]
118     assert cgl[1] == 0.0
119     assert cgl[2] == 0.0
120     assert math.isclose(cgl[3], 800 / OB, rel_tol=1e-12)
121     assert math.isclose(cgl[4], 800 / OB, rel_tol=1e-12) # sticky
122     assert math.isclose(cgl[5], 800 / OB, rel_tol=1e-12)
123
124
125 def test_cum_prepay(metrics_fixture: VintageMetrics) -> None:
126     cpp = metrics_fixture.cum_prepay().loc[V]
127     assert cpp[1] == 0.0
128     assert cpp[2] == 0.0
129     # B prepays 700 of unscheduled (800 paid - 100 scheduled) at MOB 3
130     assert math.isclose(cpp[3], 700 / OB, rel_tol=1e-12)
131     assert math.isclose(cpp[4], 700 / OB, rel_tol=1e-12)
132     assert math.isclose(cpp[5], 700 / OB, rel_tol=1e-12)
133
134
135 def test_smm_excludes_defaults_from_denominator(metrics_fixture: VintageMetrics) -> None:
136     smm = metrics_fixture.smm().loc[V]
137     # MOB 3: numerator = 700 (B's prepay), denom = BOP_perf 2400 - sched_perf 200 - default_bal 800 = 1400
138     assert math.isclose(smm[3], 700 / 1400, rel_tol=1e-12)
139     # Other months: no prepay → 0
140     assert smm[1] == 0.0
141     assert smm[2] == 0.0
142     assert smm[4] == 0.0
143     assert smm[5] == 0.0
144
145
146 def test_cpr_annualization(metrics_fixture: VintageMetrics) -> None:
147     cpr = metrics_fixture.cpr().loc[V]

```

```

148     smm3 = 700 / 1400
149     assert math.isclose(cpr[3], 1 - (1 - smm3) ** 12, rel_tol=1e-12)
150
151
152 def test_mdr_uses_performing_bop_denominator(metrics_fixture: VintageMetrics) -> None:
153     mdr = metrics_fixture.mdr().loc[V]
154     # MOB 3: default_bal 800 / BOP_perf 2400 = 1/3
155     assert math.isclose(mdr[3], 800 / 2400, rel_tol=1e-12)
156     assert mdr[1] == 0.0
157     assert mdr[2] == 0.0
158
159
160 def test_cdr_annualization(metrics_fixture: VintageMetrics) -> None:
161     cdr = metrics_fixture.cdr().loc[V]
162     mdr3 = 800 / 2400
163     assert math.isclose(cdr[3], 1 - (1 - mdr3) ** 12, rel_tol=1e-12)
164
165
166 def test_wa_coupon_constant_when_all_loans_share_apr(metrics_fixture: VintageMetrics) -> None:
167     wac = metrics_fixture.wa_coupon().loc[V]
168     # All performing-at-BOP loans share apr=0.12, so WAC is exactly 0.12 wherever defined
169     for mob in [1, 2, 3, 4, 5]:
170         assert math.isclose(wac[mob], 0.12, rel_tol=1e-12)
171
172
173 def test_curve_dispatch_and_listing(metrics_fixture: VintageMetrics) -> None:
174     assert "cum_gross_loss" in metrics_fixture.available_metrics()
175     direct = metrics_fixture.pool_factor()
176     via = metrics_fixture.curve("pool_factor")
177     pd.testing.assert_frame_equal(direct, via)
178     with pytest.raises(ValueError, match="Unknown metric"):
179         metrics_fixture.curve("not_a_metric")
180
181
182 def test_truncation_nans_beyond_max_observable_mob(metrics_fixture: VintageMetrics) -> None:
183     pf = metrics_fixture.pool_factor()
184     # Vintage's max observable MOB in this fixture is 5; cells beyond should be absent.
185     assert pf.columns.max() == 5
186     # Within columns, all entries should be non-NaN (vintage is observable for all 5 MOB)
187     assert pf.loc[V].notna().all()
188
189
190 # ----- sanity on real tape
191
192 REPO = Path(__file__).resolve().parents[1]
193 TAPE = REPO / "examples" / "sample_tape.csv"
194 SCHEMA = REPO / "examples" / "schema.yaml"
195
196
197 @pytest.fixture(scope="module")
198 def real_tape() -> LoanTape:
199     return LoanTape.from_csv(TAPE, schema=SCHEMA)
200
201
202 def test_real_tape_metrics_smoke(real_tape: LoanTape) -> None:
203     seg = Segment.from_tape(real_tape, query=None, label="all")
204     m = VintageMetrics.from_segment(seg, DefaultPolicy.charge_off_field(), vintage_freq="Q")
205     pf = m.pool_factor()
206     cgl = m.cum_gross_loss()
207     cpr = m.cpr()
208     # Pool factor starts near 1 at MOB 1 and only declines monotonically per vintage
209     for v in pf.index:
210         row = pf.loc[v].dropna()
211         assert row.iloc[0] <= 1.0 and row.iloc[0] > 0.9, (v, row.iloc[0])
212         # Allow tiny float wiggle: assert non-increasing within 1e-9
213         diffs = row.diff().dropna()
214         assert (diffs <= 1e-9).all(), (v, diffs[diffs > 1e-9])
215     # Cum loss is non-decreasing per vintage
216     for v in cgl.index:
217         row = cgl.loc[v].dropna()
218         diffs = row.diff().dropna()
219         assert (diffs >= -1e-12).all()
220     # CPR is finite and in [0, 1] where defined
221     finite = cpr.stack().dropna()
222     assert (finite >= -1e-9).all() and (finite <= 1.0 + 1e-9).all()

```

## tests/test\_segment.py

```
1  """Tests for Segment / SegmentSet filtering."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6
7  import pytest
8
9  from vintage_curves import LoanTape, Segment, SegmentSet
10 from vintage_curves.schema import CanonicalColumns
11 from vintage_curves.segment import SegmentQueryError
12
13 REPO = Path(__file__).resolve().parents[1]
14 TAPE = REPO / "examples" / "sample_tape.csv"
15 SCHEMA = REPO / "examples" / "schema.yaml"
16
17
18 @pytest.fixture(scope="module")
19 def tape() -> LoanTape:
20     return LoanTape.from_csv(TAPE, schema=SCHEMA)
21
22
23 def test_segment_filters_loans_and_perf(tape: LoanTape) -> None:
24     seg = Segment.from_tape(tape, "fico >= 720 and fico <= 759", label="prime")
25     assert seg.label == "prime"
26     assert seg.n_loans > 0
27     assert seg.n_loans < tape.n_loans
28     # All matched loans satisfy the predicate
29     assert seg.loans[CanonicalColumns.FICO].between(720, 759).all()
30     # Perf rows belong only to matched loans
31     assert seg.perf[CanonicalColumns.LOAN_ID].isin(
32         seg.loans[CanonicalColumns.LOAN_ID]
33     ).all()
34
35
36 def test_empty_query_returns_full_tape(tape: LoanTape) -> None:
37     seg = Segment.from_tape(tape, None, label="all")
38     assert seg.n_loans == tape.n_loans
39     assert seg.n_loan_months == tape.n_loan_months
40
41
42 def test_bad_query_raises(tape: LoanTape) -> None:
43     with pytest.raises(SegmentQueryError):
44         Segment.from_tape(tape, "no_such_column == 1", label="bad")
45
46
47 def test_segment_set_iteration(tape: LoanTape) -> None:
48     sset = SegmentSet.from_tape(tape, [
49         ("super_prime", "fico >= 760"),
50         ("prime", "fico >= 700 and fico < 760"),
51         ("subprime", "fico < 700"),
52     ])
53     assert len(sset) == 3
54     labels = [s.label for s in sset]
55     assert labels == ["super_prime", "prime", "subprime"]
56     # Partition (disjoint + covers all FICO-bearing loans)
57     total = sum(s.n_loans for s in sset)
58     assert total == tape.n_loans
```

## tests/test\_aggregate.py

```
1  """Tests for aggregate_curves under all three weighting schemes."""
2
3  from __future__ import annotations
4
5  import math
6  from pathlib import Path
7
8  import numpy as np
9  import pandas as pd
10 import pytest
11
12 from vintage_curves import (
13     DefaultPolicy,
14     LoanTape,
15     Segment,
16     SchemaMapper,
17     VintageMetrics,
18     aggregate_curves,
19 )
20 from vintage_curves.schema import CanonicalColumns
21
22
23 # Reuse the 3-loan analytical fixture's idiom but with two vintages so
24 # aggregation across vintages is meaningful.
25
26 CANONICAL_MAPPER = SchemaMapper(mapping={c: c for c in [
27     "loan_id", "origination_date", "original_balance", "apr", "original_term",
28     "as_of_date", "outstanding_balance", "scheduled_principal", "principal_payment",
29     "interest_payment", "days_past_due", "charge_off_date", "charge_off_amount", "fico",
30 ]})
31
32
33 def _row(loan_id, as_of, eop, sched, prin_pay, dpd, co_date=None, co_amt=0.0):
34     return {
35         "loan_id": loan_id, "as_of_date": as_of, "outstanding_balance": eop,
36         "scheduled_principal": sched, "principal_payment": prin_pay,
37         "interest_payment": 0.0, "days_past_due": dpd,
38         "charge_off_date": co_date, "charge_off_amount": co_amt,
39     }
40
41
42 @pytest.fixture
43 def two_vintage_metrics() -> VintageMetrics:
44     # Vintage 2024Q1: 1 loan, OB=1000, pays scheduled for 3 months
45     # Vintage 2024Q2: 1 loan, OB=2000, defaults at MOB 2 (CO=1900)
46     loans = pd.DataFrame([
47         {"loan_id": "Q1", "origination_date": "2024-01-15",
48          "original_balance": 1000.0, "apr": 0.12, "original_term": 10, "fico": 700},
49         {"loan_id": "Q2", "origination_date": "2024-04-15",
50          "original_balance": 2000.0, "apr": 0.12, "original_term": 10, "fico": 700},
51     ])
52     perf = pd.DataFrame([
53         _row("Q1", "2024-01-31", 900.0, 100.0, 100.0, 0),
54         _row("Q1", "2024-02-29", 800.0, 100.0, 100.0, 0),
55         _row("Q1", "2024-03-31", 700.0, 100.0, 100.0, 0),
56         _row("Q2", "2024-04-30", 1900.0, 100.0, 100.0, 0),
57         _row("Q2", "2024-05-31", 0.0, 100.0, 0.0, 120,
58              co_date="2024-05-31", co_amt=1900.0),
59     ])
60     raw = perf.merge(loans, on="loan_id", how="left")
61     tape = LoanTape.from_dataframe(raw, CANONICAL_MAPPER)
62     seg = Segment.from_tape(tape, query=None, label="all")
63     return VintageMetrics.from_segment(seg, DefaultPolicy.charge_off_field(), vintage_freq="Q")
64
65
66 def test_n_vintages_reflects_support(two_vintage_metrics: VintageMetrics) -> None:
67     agg = aggregate_curves(two_vintage_metrics, ["pool_factor"], weighting="original")
68     # Q1 observable for MOB 1-3; Q2 observable for MOB 1-2
69     assert agg.loc[1, "n_vintages"] == 2
70     assert agg.loc[2, "n_vintages"] == 2
71     assert agg.loc[3, "n_vintages"] == 1
72
```

```

73
74 def test_original_weighted_pool_factor(two_vintage_metrics: VintageMetrics) -> None:
75     agg = aggregate_curves(two_vintage_metrics, ["pool_factor"], weighting="original")
76     # MOB 1: Q1 EOP 900 / OB 1000 = 0.9, Q2 EOP 1900 / OB 2000 = 0.95.
77     # OB-weighted: (1000*0.9 + 2000*0.95) / 3000 = (900 + 1900)/3000 = 2800/3000.
78     assert math.isclose(agg.loc[1, "pool_factor"], 2800 / 3000, rel_tol=1e-12)
79     # MOB 2: Q1 PF=800/1000=0.8, Q2 PF=0/2000=0 (charged off).
80     # OB-weighted: (1000*0.8 + 2000*0)/3000 = 800/3000.
81     assert math.isclose(agg.loc[2, "pool_factor"], 800 / 3000, rel_tol=1e-12)
82     # MOB 3: only Q1 observable; agg = 0.7
83     assert math.isclose(agg.loc[3, "pool_factor"], 0.7, rel_tol=1e-12)
84
85
86 def test_equal_weighted_differs_from_original(two_vintage_metrics: VintageMetrics) -> None:
87     agg_eq = aggregate_curves(two_vintage_metrics, ["pool_factor"], weighting="equal")
88     agg_ob = aggregate_curves(two_vintage_metrics, ["pool_factor"], weighting="original")
89     # MOB 1 equal: (0.9 + 0.95) / 2 = 0.925
90     assert math.isclose(agg_eq.loc[1, "pool_factor"], 0.925, rel_tol=1e-12)
91     assert agg_eq.loc[1, "pool_factor"] != agg_ob.loc[1, "pool_factor"]
92
93
94 def test_cum_gross_loss_aggregation(two_vintage_metrics: VintageMetrics) -> None:
95     agg = aggregate_curves(two_vintage_metrics, ["cum_gross_loss"], weighting="original")
96     # MOB 1: no loss anywhere → 0
97     assert agg.loc[1, "cum_gross_loss"] == 0.0
98     # MOB 2: Q2 loss=1900/2000=0.95, Q1 loss=0. OB-weighted = (1000*0 + 2000*0.95)/3000 = 1900/3000
99     assert math.isclose(agg.loc[2, "cum_gross_loss"], 1900 / 3000, rel_tol=1e-12)
100    # MOB 3: only Q1 observable, still 0 loss
101    assert agg.loc[3, "cum_gross_loss"] == 0.0
102
103
104 def test_flow_metric_sums_dollars(two_vintage_metrics: VintageMetrics) -> None:
105     # MOB 2 CDR: default_bal Q2 = 1900 (Q2 BOP at MOB 2), Q1 default_bal = 0.
106     # BOP_perf Q2 = 1900, Q1 = 900. Aggregate MDR = 1900 / 2800.
107     agg = aggregate_curves(two_vintage_metrics, ["mdr", "cdr"], weighting="original")
108     mdr2 = 1900 / 2800
109     assert math.isclose(agg.loc[2, "mdr"], mdr2, rel_tol=1e-12)
110     assert math.isclose(agg.loc[2, "cdr"], 1 - (1 - mdr2) ** 12, rel_tol=1e-12)
111
112
113 def test_unknown_metric_and_weighting_raise(two_vintage_metrics: VintageMetrics) -> None:
114     with pytest.raises(ValueError, match="Unknown metric"):
115         aggregate_curves(two_vintage_metrics, ["nope"], weighting="original")
116     with pytest.raises(ValueError, match="weighting"):
117         aggregate_curves(two_vintage_metrics, ["pool_factor"], weighting="bogus") # type: ignore[arg-type]
118
119
120 # ----- real-tape smoke
121
122 REPO = Path(__file__).resolve().parents[1]
123 TAPE = REPO / "examples" / "sample_tape.csv"
124 SCHEMA = REPO / "examples" / "schema.yaml"
125
126
127 @pytest.fixture(scope="module")
128 def real_metrics() -> VintageMetrics:
129     tape = LoanTape.from_csv(TAPE, schema=SCHEMA)
130     seg = Segment.from_tape(tape, query=None, label="all")
131     return VintageMetrics.from_segment(seg, DefaultPolicy.charge_off_field(), vintage_freq="Q")
132
133
134 def test_real_tape_aggregate_all_weightings(real_metrics: VintageMetrics) -> None:
135     metrics_to_request = ["cum_gross_loss", "cum_prepay", "pool_factor", "smm", "cpr", "mdr", "cdr"]
136     for w in ("original", "beginning", "equal"):
137         agg = aggregate_curves(real_metrics, metrics_to_request, weighting=w)
138         assert "n_vintages" in agg.columns
139         # n_vintages monotonically non-increasing as MOB grows past max observable
140         nv = agg["n_vintages"].to_numpy()
141         assert (np.diff(nv) <= 0).all(), f"n_vintages should not grow under {w}: {nv}"
142         # CPR / CDR in [0, 1]
143         for col in ("cpr", "cdr"):
144             vals = agg[col].dropna()
145             assert (vals >= -1e-9).all() and (vals <= 1.0 + 1e-9).all()

```

## tests/test\_rolls.py

```
1  """Tests for delinquency bucket assignment and roll-rate transition matrices."""
2
3  from __future__ import annotations
4
5  import math
6  from pathlib import Path
7
8  import numpy as np
9  import pandas as pd
10 import pytest
11
12 from vintage_curves import DefaultPolicy, LoanTape, Segment, SchemaMapper, VintageMetrics
13 from vintage_curves.default_policy import IS_DEFAULTED
14 from vintage_curves.mob import add_mob_and_vintage
15 from vintage_curves.rolls import (
16     BUCKET,
17     CO_BUCKET,
18     DEFAULT_BOUNDARIES,
19     assign_buckets,
20     transition_matrix,
21 )
22 from vintage_curves.schema import CanonicalColumns
23
24
25 CANONICAL_MAPPER = SchemaMapper(mapping={c: c for c in [
26     "loan_id", "origination_date", "original_balance", "apr", "original_term",
27     "as_of_date", "outstanding_balance", "scheduled_principal", "principal_payment",
28     "interest_payment", "days_past_due", "charge_off_date", "charge_off_amount", "fico",
29 ]})
30
31
32 def _row(loan_id, as_of, eop, sched, prin_pay, dpd, co_date=None, co_amt=0.0):
33     return {
34         "loan_id": loan_id, "as_of_date": as_of, "outstanding_balance": eop,
35         "scheduled_principal": sched, "principal_payment": prin_pay,
36         "interest_payment": 0.0, "days_past_due": dpd,
37         "charge_off_date": co_date, "charge_off_amount": co_amt,
38     }
39
40
41 @pytest.fixture
42 def flagged_perf() -> pd.DataFrame:
43     """Three loans with different roll behaviors."""
44     loans = pd.DataFrame([
45         {"loan_id": "A", "origination_date": "2024-01-15",
46          "original_balance": 1000.0, "apr": 0.12, "original_term": 10, "fico": 700},
47         {"loan_id": "B", "origination_date": "2024-01-15",
48          "original_balance": 1000.0, "apr": 0.12, "original_term": 10, "fico": 700},
49         {"loan_id": "C", "origination_date": "2024-01-15",
50          "original_balance": 1000.0, "apr": 0.12, "original_term": 10, "fico": 700},
51     ])
52     perf = pd.DataFrame([
53         # A: stays current
54         _row("A", "2024-01-31", 900.0, 100.0, 100.0, 0),
55         _row("A", "2024-02-29", 800.0, 100.0, 100.0, 0),
56         _row("A", "2024-03-31", 700.0, 100.0, 100.0, 0),
57         # B: Current → 30-59 → 60-89 → cures back to Current
58         _row("B", "2024-01-31", 900.0, 100.0, 100.0, 0),
59         _row("B", "2024-02-29", 900.0, 100.0, 0.0, 30),
60         _row("B", "2024-03-31", 900.0, 100.0, 0.0, 60),
61         _row("B", "2024-04-30", 800.0, 100.0, 200.0, 0),
62         # C: 30-59 → CO via DPD on month 3 (with charge_off_date populated)
63         _row("C", "2024-01-31", 900.0, 100.0, 100.0, 30),
64         _row("C", "2024-02-29", 900.0, 100.0, 0.0, 90),
65         _row("C", "2024-03-31", 0.0, 100.0, 0.0, 120,
66              co_date="2024-03-31", co_amt=900.0),
67     ])
68     raw = perf.merge(loans, on="loan_id", how="left")
69     tape = LoanTape.from_dataframe(raw, CANONICAL_MAPPER)
70     seg = Segment.from_tape(tape, query=None, label="all")
71     perf_v = add_mob_and_vintage(seg.perf, seg.loans, vintage_freq="Q")
72     return DefaultPolicy.charge_off_field().apply(perf_v)
```

```

73
74
75 def test_bucket_assignment_basic(flagged_perf: pd.DataFrame) -> None:
76     buckets = assign_buckets(flagged_perf)
77     flagged_perf = flagged_perf.assign(b=buckets.values)
78     # Loan A always Current
79     assert (flagged_perf[flagged_perf["loan_id"] == "A"]["b"] == "Current").all()
80     # Loan B sequence
81     seq_b = flagged_perf[flagged_perf["loan_id"] == "B"].sort_values("mob")["b"].tolist()
82     assert seq_b == ["Current", "30-59", "60-89", "Current"]
83     # Loan C ends in C0 (sticky from MOB 3 onward – only MOB 3 row here)
84     seq_c = flagged_perf[flagged_perf["loan_id"] == "C"].sort_values("mob")["b"].tolist()
85     assert seq_c == ["30-59", "90+", "C0"]
86
87
88 def test_co_is_sticky_overrides_dpd() -> None:
89     # Hand-build a row where DPD is 0 but is_defaulted is True (post-C0 survivor row)
90     df = pd.DataFrame({
91         "loan_id": ["X", "X"],
92         "mob": [1, 2],
93         CanonicalColumns.DAYS_PAST_DUE: [0, 0],
94         IS_DEFAULTED: [False, True],
95     })
96     buckets = assign_buckets(df)
97     assert buckets.tolist() == ["Current", "C0"]
98
99
100 def test_transition_matrix_counts(flagged_perf: pd.DataFrame) -> None:
101     tm = transition_matrix(flagged_perf, weight="count")
102     # Rows are from-bucket, columns are to-bucket. Rows sum to 1.
103     row_sums = tm.sum(axis=1)
104     for label, s in row_sums.items():
105         # Buckets with no observed transitions sum to 0; the rest sum to 1.
106         assert s in (0.0, pytest.approx(1.0, abs=1e-12)), (label, s)
107
108     # Specific observed transitions:
109     # Current→Current: A has 2 such (mob 1→2, 2→3), B has 1 (mob 4 is last so no next), so 2 + ...
110     # Adjacency requires next_mob - mob == 1. Last row of each loan is dropped.
111     # A: (Current→Current) x2 ; B: (Current→30-59), (30-59→60-89), (60-89→Current); C: (30-59→90+), (90+→C0)
112     # So Current→Current = 2, Current→30-59 = 1, 30-59→60-89 = 1, 30-59→90+ = 1,
113     # 60-89→Current = 1, 90+→C0 = 1.
114     # Row totals: Current=3, 30-59=2, 60-89=1, 90+=1
115     assert math.isclose(tm.loc["Current", "Current"], 2 / 3, rel_tol=1e-12)
116     assert math.isclose(tm.loc["Current", "30-59"], 1 / 3, rel_tol=1e-12)
117     assert math.isclose(tm.loc["30-59", "60-89"], 0.5, rel_tol=1e-12)
118     assert math.isclose(tm.loc["30-59", "90+"], 0.5, rel_tol=1e-12)
119     assert math.isclose(tm.loc["60-89", "Current"], 1.0, rel_tol=1e-12)
120     assert math.isclose(tm.loc["90+", "C0"], 1.0, rel_tol=1e-12)
121
122
123 def test_transition_matrix_balance_weight(flagged_perf: pd.DataFrame) -> None:
124     tm = transition_matrix(flagged_perf, weight="balance")
125     # Each non-zero row still sums to 1
126     row_sums = tm.sum(axis=1)
127     for label, s in row_sums.items():
128         assert s in (0.0, pytest.approx(1.0, abs=1e-12)), (label, s)
129
130
131 def test_per_mob_transition_matrix(flagged_perf: pd.DataFrame) -> None:
132     tm = transition_matrix(flagged_perf, per_mob=True)
133     assert tm.index.names == ["mob", "BUCKET"]
134     # Each (mob, from_bucket) row that has any flow normalizes to 1
135     for idx, row in tm.iterrows():
136         total = row.sum()
137         assert total in (0.0, pytest.approx(1.0, abs=1e-12)), (idx, total)
138
139
140 def test_invalid_weight_raises(flagged_perf: pd.DataFrame) -> None:
141     with pytest.raises(ValueError, match="weight"):
142         transition_matrix(flagged_perf, weight="weird") # type: ignore[arg-type]
143
144
145 # ----- real-tape smoke
146
147 REPO = Path(__file__).resolve().parents[1]

```



```

148 TAPE = REPO / "examples" / "sample_tape.csv"
149 SCHEMA = REPO / "examples" / "schema.yaml"
150
151
152 def test_rolls_on_real_tape() -> None:
153     tape = LoanTape.from_csv(TAPE, schema=SCHEMA)
154     seg = Segment.from_tape(tape, query=None, label="all")
155     perf_v = add_mob_and_vintage(seg.perf, seg.loans, vintage_freq="Q")
156     flagged = DefaultPolicy.charge_off_field().apply(perf_v)
157     tm = transition_matrix(flagged, weight="count")
158     # Most loans stay Current – diagonal Current→Current is large
159     assert tm.loc["Current", "Current"] > 0.9
160     # Once defaulted, sticky → CO→CO ≈ 1
161     if tm.loc[CO_BUCKET].sum() > 0:
162         assert math.isclose(tm.loc[CO_BUCKET, CO_BUCKET], 1.0, rel_tol=1e-9)

```

## tests/test\_summary.py

```
1  """Tests for per-vintage summary scalars."""
2
3  from __future__ import annotations
4
5  import math
6
7  import pandas as pd
8  import pytest
9
10 from vintage_curves import (
11     DefaultPolicy,
12     LoanTape,
13     Segment,
14     SchemaMapper,
15     VintageMetrics,
16     vintage_summary,
17 )
18
19
20 CANONICAL_MAPPER = SchemaMapper(mapping={c: c for c in [
21     "loan_id", "origination_date", "original_balance", "apr", "original_term",
22     "as_of_date", "outstanding_balance", "scheduled_principal", "principal_payment",
23     "interest_payment", "days_past_due", "charge_off_date", "charge_off_amount", "fico",
24 ]})
25
26
27 def _row(loan_id, as_of, eop, sched, prin_pay, dpd, co_date=None, co_amt=0.0):
28     return {
29         "loan_id": loan_id, "as_of_date": as_of, "outstanding_balance": eop,
30         "scheduled_principal": sched, "principal_payment": prin_pay,
31         "interest_payment": 0.0, "days_past_due": dpd,
32         "charge_off_date": co_date, "charge_off_amount": co_amt,
33     }
34
35
36 @pytest.fixture
37 def mixed_apr_metrics() -> VintageMetrics:
38     # Two loans in the same vintage with different APR / OB to test WAC weighting.
39     # Loan A: OB=1000, APR=0.10, observed 3 months (scheduled $100/mo, no events)
40     # Loan B: OB=3000, APR=0.20, observed 3 months (scheduled $300/mo, no events)
41     loans = pd.DataFrame([
42         {"loan_id": "A", "origination_date": "2024-01-15",
43          "original_balance": 1000.0, "apr": 0.10, "original_term": 10, "fico": 700},
44         {"loan_id": "B", "origination_date": "2024-01-15",
45          "original_balance": 3000.0, "apr": 0.20, "original_term": 10, "fico": 700},
46     ])
47     perf = pd.DataFrame([
48         _row("A", "2024-01-31", 900.0, 100.0, 100.0, 0),
49         _row("A", "2024-02-29", 800.0, 100.0, 100.0, 0),
50         _row("A", "2024-03-31", 700.0, 100.0, 100.0, 0),
51         _row("B", "2024-01-31", 2700.0, 300.0, 300.0, 0),
52         _row("B", "2024-02-29", 2400.0, 300.0, 300.0, 0),
53         _row("B", "2024-03-31", 2100.0, 300.0, 300.0, 0),
54     ])
55     raw = perf.merge(loans, on="loan_id", how="left")
56     tape = LoanTape.from_dataframe(raw, CANONICAL_MAPPER)
57     seg = Segment.from_tape(tape, query=None, label="all")
58     return VintageMetrics.from_segment(seg, DefaultPolicy.charge_off_field(), vintage_freq="Q")
59
60
61 def test_summary_scalars(mixed_apr_metrics: VintageMetrics) -> None:
62     s = vintage_summary(mixed_apr_metrics)
63     v = pd.Period("2024Q1")
64     row = s.loc[v]
65     assert row["original_count"] == 2
66     assert math.isclose(row["original_balance"], 4000.0, rel_tol=1e-12)
67     # OB-weighted WAC: (1000*0.10 + 3000*0.20) / 4000 = 700/4000 = 0.175
68     assert math.isclose(row["wa_coupon"], 0.175, rel_tol=1e-12)
69     # Pool factor at MOB 3: (700 + 2100) / 4000 = 0.7
70     assert math.isclose(row["pool_factor_latest"], 0.7, rel_tol=1e-12)
71     # WAL realized: 100*1+100*2+100*3 = 600 for A; 300*1+300*2+300*3 = 1800 for B.
72     # Combined weighted: (1*100+2*100+3*100) + (1*300+2*300+3*300) = 600 + 1800 = 2400
```

```

73     # Principal removed: 300 + 900 = 1200. WAL = 2400/1200 = 2.0
74     assert math.isclose(row["wal_realized"], 2.0, rel_tol=1e-12)
75     # WAL scheduled exists and is positive
76     assert row["wal_scheduled"] > 0
77     assert row["wal_scheduled"] <= 10 # bounded by original term
78
79
80 def test_wal_scheduled_for_zero_rate_loan() -> None:
81     # With APR=0 and term=12, level pay is equal principal each month → WAL = (n+1)/2 = 6.5
82     loans = pd.DataFrame([
83         {"loan_id": "Z", "origination_date": "2024-01-15",
84          "original_balance": 1200.0, "apr": 0.0, "original_term": 12, "fico": 700,
85         })
86     perf = pd.DataFrame([
87         _row("Z", "2024-01-31", 1100.0, 100.0, 100.0, 0),
88     ])
89     raw = perf.merge(loans, on="loan_id", how="left")
90     tape = LoanTape.from_dataframe(raw, CANONICAL_MAPPER)
91     seg = Segment.from_tape(tape, query=None, label="all")
92     m = VintageMetrics.from_segment(seg, DefaultPolicy.charge_off_field(), vintage_freq="Q")
93     s = vintage_summary(m)
94     assert math.isclose(s.iloc[0]["wal_scheduled"], 6.5, rel_tol=1e-12)

```

## tests/test\_end\_to\_end.py

```
1  """End-to-end test against the example sample tape via VintageAnalyzer."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6
7  import matplotlib
8
9  matplotlib.use("Agg") # headless backend for CI
10
11 import pandas as pd
12 import pytest
13
14 from vintage_curves import DefaultPolicy, VintageAnalyzer
15
16 REPO = Path(__file__).resolve().parents[1]
17 TAPE = REPO / "examples" / "sample_tape.csv"
18 SCHEMA = REPO / "examples" / "schema.yaml"
19
20
21 @pytest.fixture(scope="module")
22 def analyzer() -> VintageAnalyzer:
23     return VintageAnalyzer.from_csv(
24         TAPE, schema=SCHEMA, vintage_freq="Q",
25         default_policy=DefaultPolicy.charge_off_field(),
26     )
27
28
29 def test_segment_curve_and_aggregate(analyzer: VintageAnalyzer) -> None:
30     seg = analyzer.segment("fico >= 700", label="prime+")
31     cgl = seg.curve("cum_gross_loss")
32     assert isinstance(cgl, pd.DataFrame)
33     assert cgl.shape[0] == 3 # three vintages
34
35     agg = seg.aggregate(["cum_gross_loss", "cpr", "cdr"], weighting="original")
36     assert {"cum_gross_loss", "cpr", "cdr", "n_vintages"}.issubset(agg.columns)
37     assert (agg["n_vintages"] >= 1).all()
38
39
40 def test_summary_and_rolls(analyzer: VintageAnalyzer) -> None:
41     seg = analyzer.all()
42     summ = seg.summary()
43     assert summ.shape == (3, 6)
44     assert (summ["original_count"] == 11_200).all()
45
46     rolls = seg.roll_rates(by_mob=False, weight="count")
47     # Each row sums to 0 or 1
48     row_sums = rolls.sum(axis=1)
49     assert ((row_sums == 0) | (row_sums.round(9) == 1)).all()
50
51
52 def test_plot_returns_figure(analyzer: VintageAnalyzer) -> None:
53     seg = analyzer.all()
54     fig = seg.plot("cum_gross_loss", include_agg=True, weighting="original")
55     assert fig is not None
56     assert len(fig.axes) == 1
57
58
59 def test_excel_export_roundtrip(tmp_path: Path, analyzer: VintageAnalyzer) -> None:
60     seg = analyzer.segment("original_term == 36", label="36mo")
61     out = seg.export_excel(
62         tmp_path / "out.xlsx",
63         metrics=["cum_gross_loss", "pool_factor", "cpr"],
64     )
65     assert out.exists()
66     book = pd.ExcelFile(out)
67     expected = {
68         "cum_gross_loss", "pool_factor", "cpr",
69         "summary", "roll_rates",
70         "agg_original", "agg_beginning", "agg_equal",
71     }
72     assert expected.issubset(set(book.sheet_names))
```

```
73
74 # Spot-check: agg_original sheet contains an n_vintages column
75 agg_sheet = pd.read_excel(out, sheet_name="agg_original", index_col=0)
76 assert "n_vintages" in agg_sheet.columns
77
78
79 def test_csv_dir_export(tmp_path: Path, analyzer: VintageAnalyzer) -> None:
80     seg = analyzer.all()
81     out_dir = seg.export_csv_dir(
82         tmp_path / "out", metrics=["cum_gross_loss", "cpr"]
83     )
84     files = {p.name for p in out_dir.iterdir()}
85     assert {"cum_gross_loss.csv", "cpr.csv", "summary.csv", "roll_rates.csv",
86            "agg_original.csv", "agg_beginning.csv", "agg_equal.csv"}.issubset(files)
```